JENKINS CICD WITH GITHUB INTEGRATION

Overview:

- This project demonstrates the implementation of a Continuous Integration and Continuous Deployment (CI/CD) pipeline using Jenkins, integrated with GitHub.

Description:

- By Continuous Integration and Continuous Deployment (CI/CD) pipeline using Jenkins, integrated with GitHub, the goal is to automate the build, test, and deployment processes, enhancing the efficiency and reliability of software development and delivery.

Prerequisites:

We must have some basic knowledge on:

1. **Continuous Integration:** Automatically build and test code changes whenever new commits are pushed to the GitHub repository.
2. **Continuous Deployment:** Deploy the application to the target environment after successful builds and tests, ensuring quick and reliable updates.
3. **Version Control:** Utilize GitHub for source code management, tracking changes, and collaboration among team members.

4. **Amazon EC2 (Elastic Compute Cloud) -** Web service which provides scalable computing capacity in the cloud.

Instance - Virtual computing environments.

5. **Containerization:** Docker allows applications to be packaged into containers, which can run consistently across different environments, whether it's a developer's local machine, a testing environment, or a production server.

6. **Webhooks :** Webhooks in Jenkins are a way to trigger Jenkins jobs (builds, pipelines, etc.) automatically in response to events from other services or applications. They are typically used to integrate Jenkins with other systems such as version control repositories, issue trackers, and continuous deployment tools.

Data Flow:

a. Developer develops the code, which is used for creation of an application and the written code is pushed into GitHub.

b. From GitHub Repository, code is taken by Devops Engineer and the role of Devops Engineer is to deploy the application code in any

Rohith Lydalla

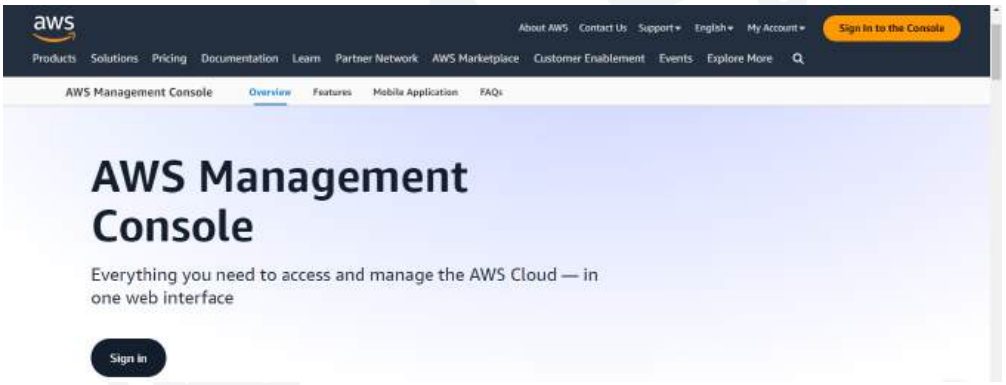
cloud environment(AWS CLOUD) to make the application accessible for the users.

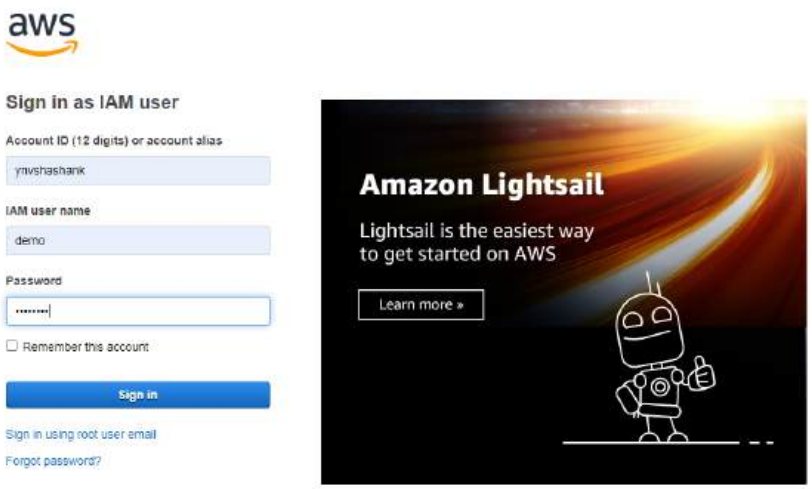
c. The Code of the application is deployed on EC2 Instance on AWS Cloud environment with the help of CI/CD process.
d. Using CI process, which is continuous integration, means whatever code is pushed into GitHub which is written by the developer will be continuously integrated.
e. Continuously integrated means whenever there are changes in the code or new versions of the application are created, using Jenkins, which is a CI tool, it will automatically integrate, builts, test and deploy the application.
f. After the application is ready and it is in the deployment state, using Docker which is a containerization tool, where DevOps Engineers use to deploy an application in isolated environments.

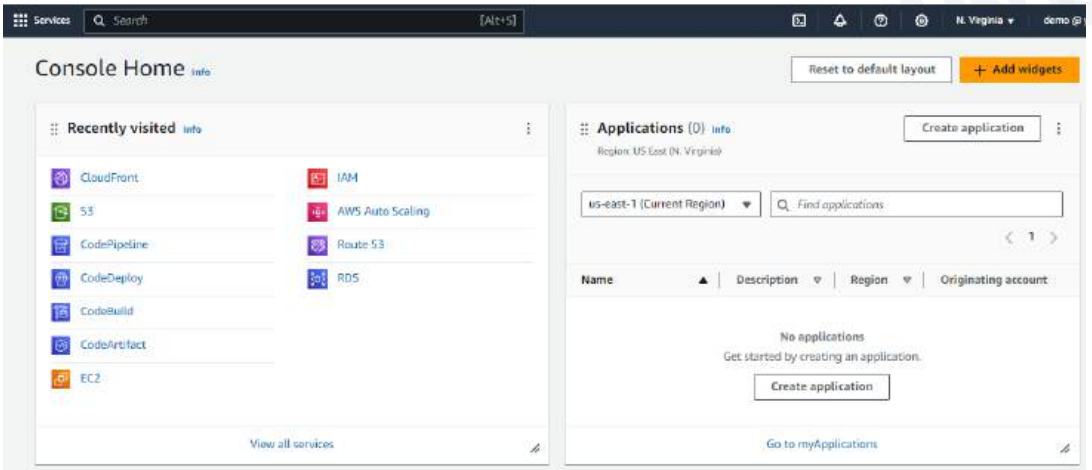
Steps:

Step 1: Login into AWS Console

1.1 Create an account and login into it by providing the credentials.

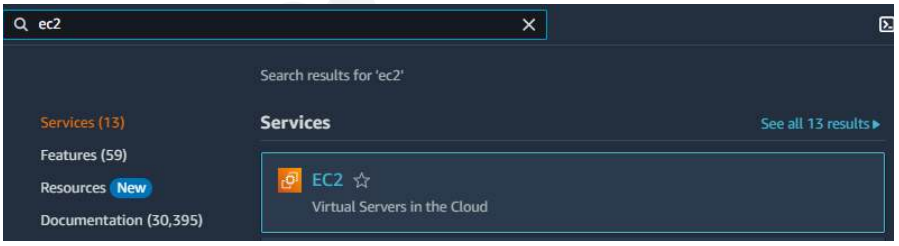




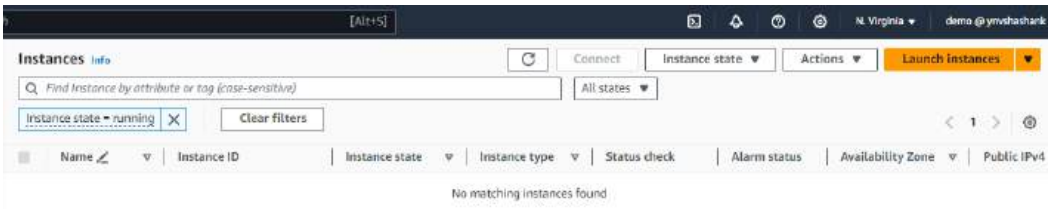


Step 2: Launch an EC2 Instance in Console

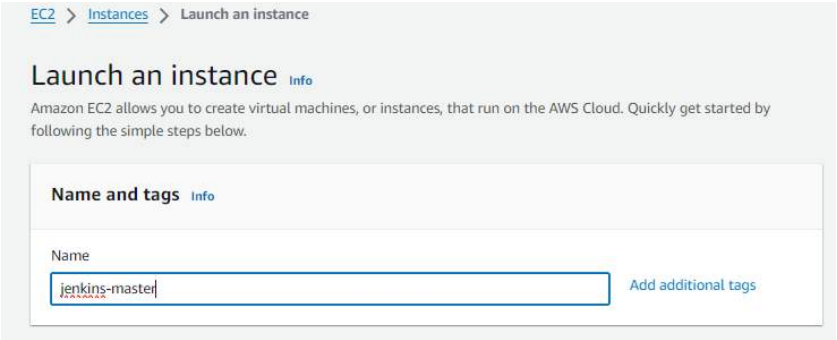
2.1 In Dashboard, search for **EC2 service,** click on **EC2** to open it.



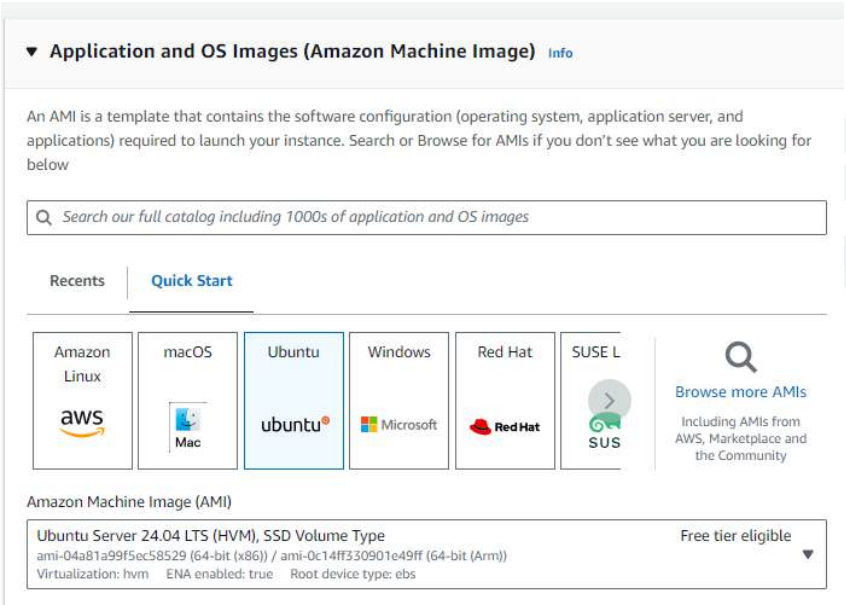
2.2 Click on **Launch instance** to configure.



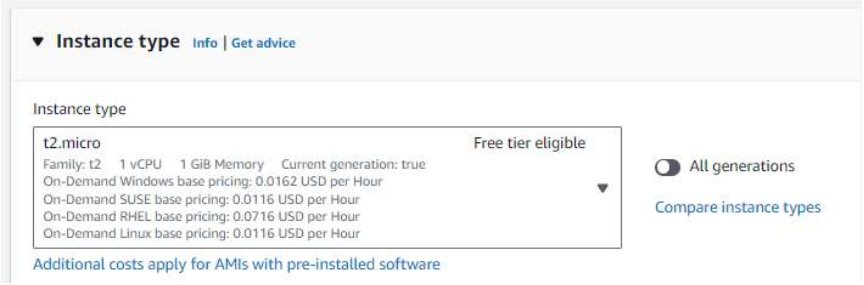
2.3 Type the **Name** which you prefer for naming the instance.



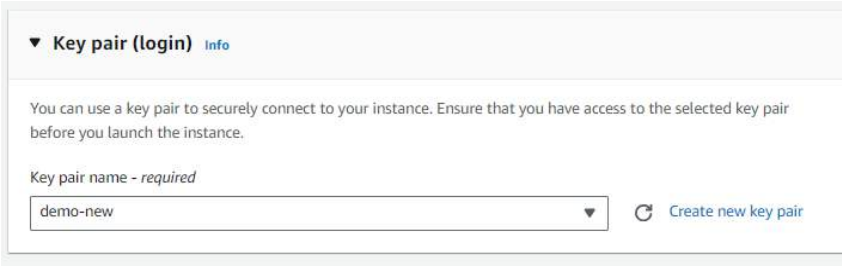
2.4 Choose the **AMI**, so here we are using **Ubuntu**.



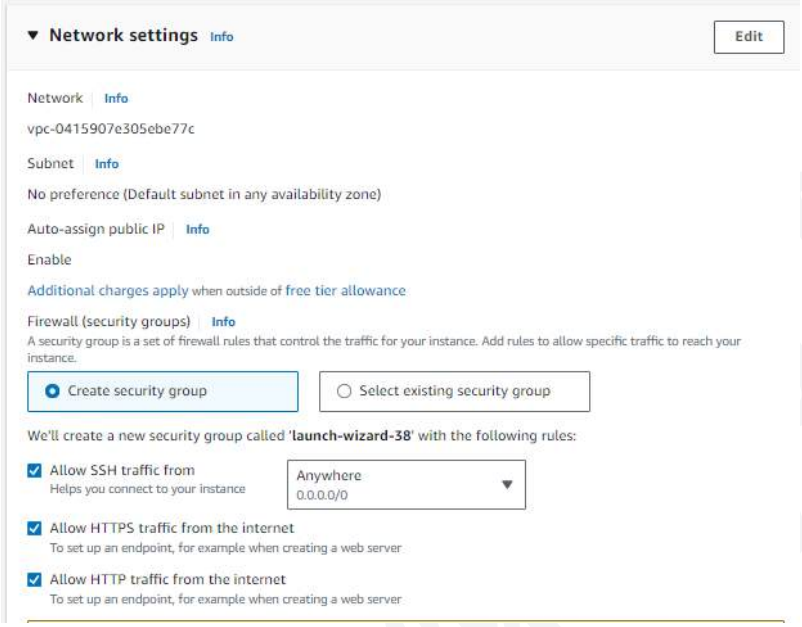
2.5 Select the instance type as **t2.micro**



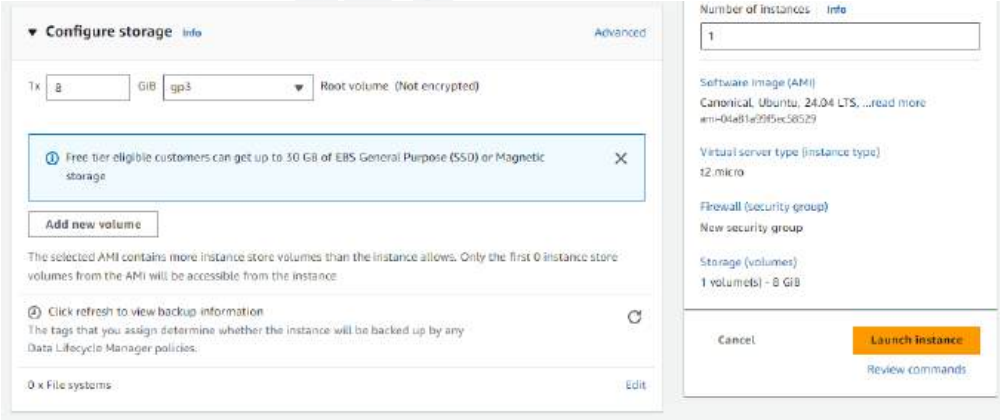
2.6 Create the **key Pair** to do that, Click on Create the **New pair key** for secure connection.

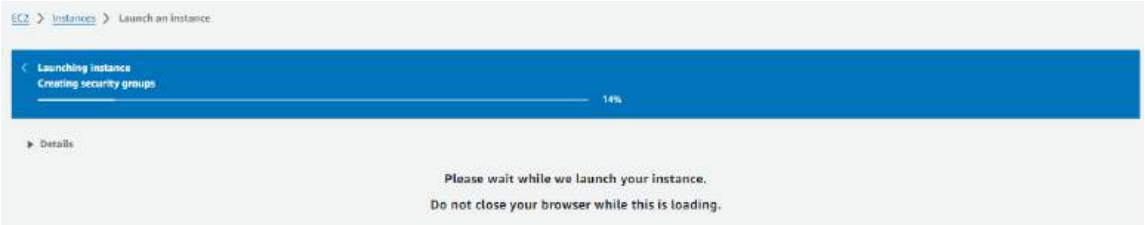


2.7 Under Network Settings, click **Edit** and then assign a **Default VPC** or you can also assign a VPC which is created by you.



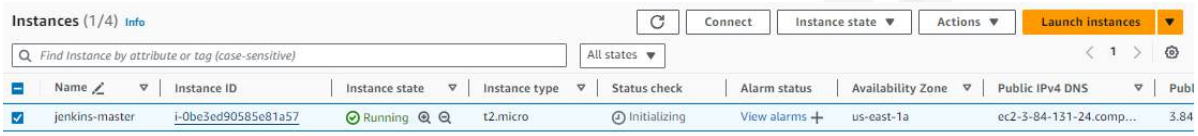
2.8 Leave the rest of the configuration as default and then click on **Launch Instance**.

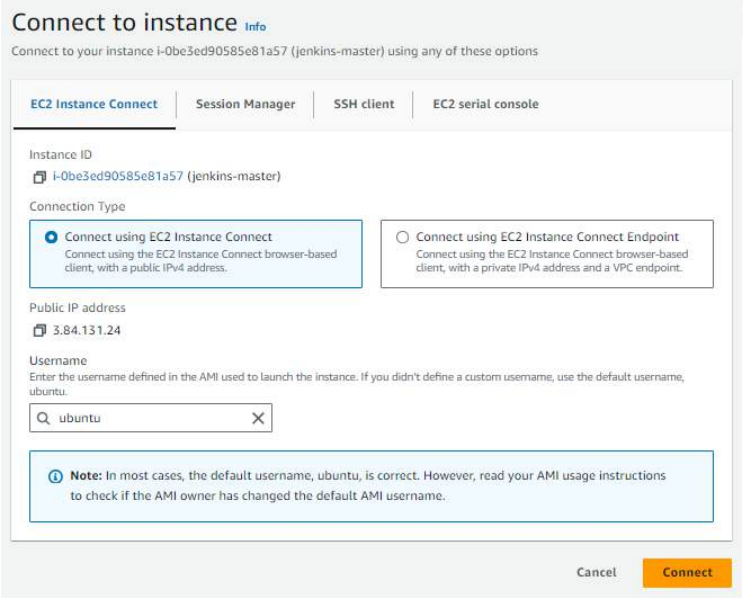




Step 3: Installation of Jenkins Using EC2 Instance

3.1 Go to the **EC2 Instance console,** select the **created EC2 Instance** and then Click on **Connect** to connect the server and install Jenkins on it.





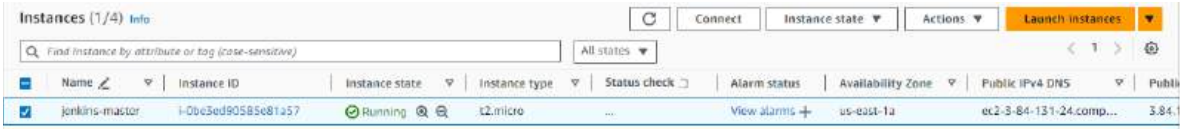
```
Welcome to Ubuntu 24.04 LTS (GNU/Linux 6.8.0-1009-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Thu Jul  4 06:53:19 UTC 2024

  System load:  0.69               Processes:             106
  Usage of /:   22.7% of 6.71GB    Users logged in:       0
  Memory usage: 22%                IPv4 address for enX0: 172.31.80.109
  Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status



The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
```

3.2 Follow the command to i**nstall Jenkins on EC2 Ubuntu server**

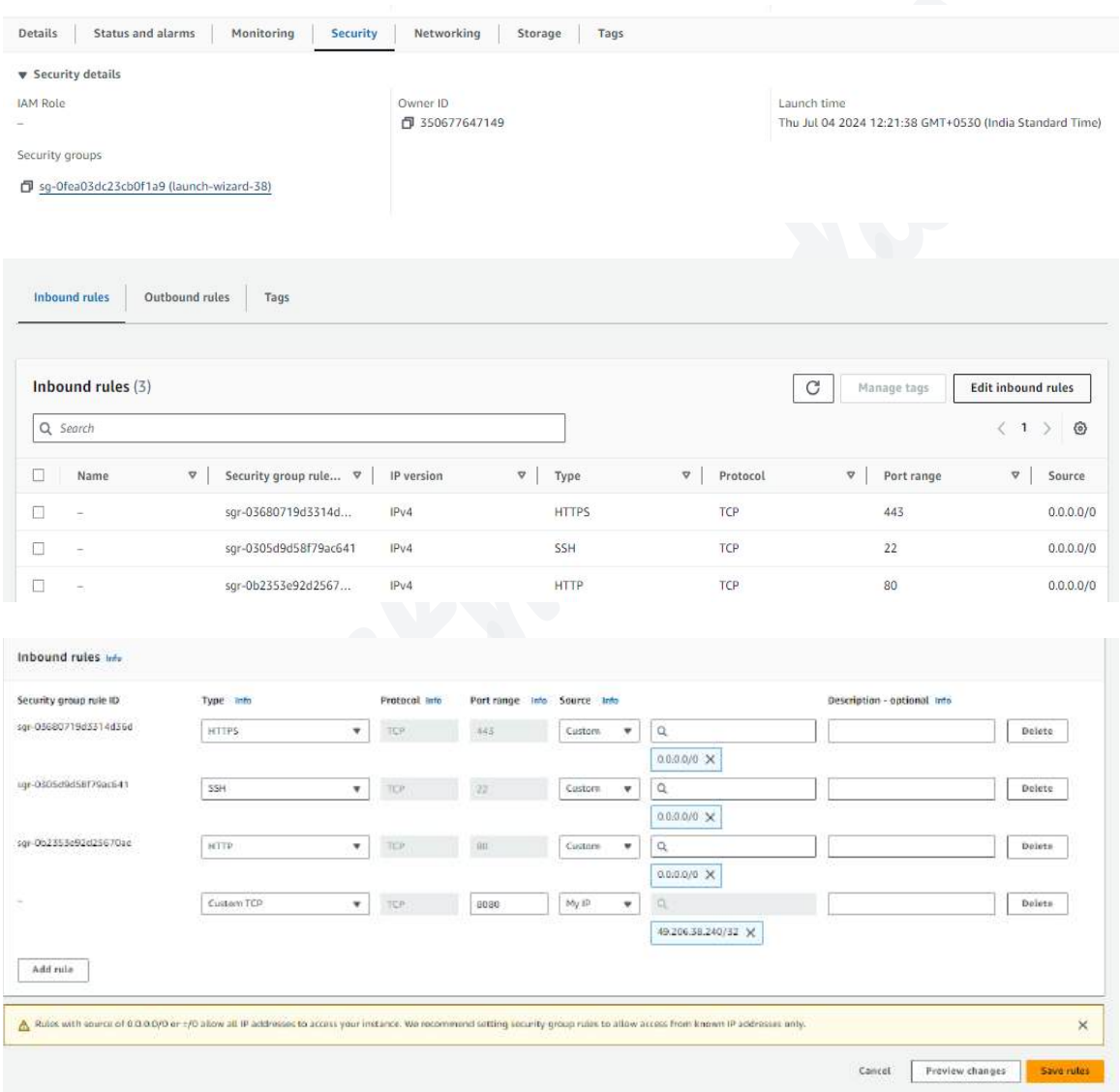
```
Step - 1 Install Java

Update your system

sudo apt update

Install java

sudo apt install openjdk-11-jre

Validate Installation

java -version

Step - 2 Install Jenkins
Just copy these commands and paste them onto your terminal.

curl -fsSL https://pkg.jenkins.io/debian/jenkins.io.key | sudo tee \   /usr/sha

echo deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] \   https://pkg.je


sudo apt-get update

sudo apt-get install jenkins

Step -3 Start jenkins

sudo systemctl enable jenkins
sudo systemctl start jenkins
sudo systemctl status jenkins
```

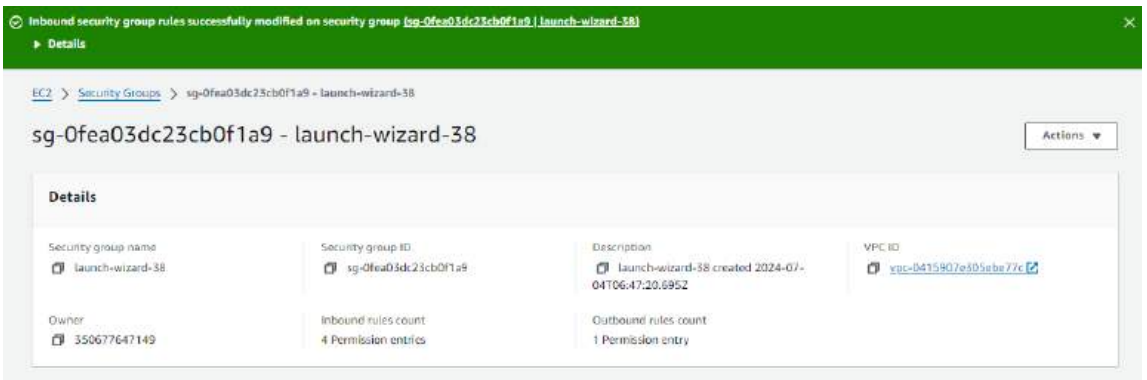
```
No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@ip-172-31-80-109:~$ jenkins --version
2.452.2
ubuntu@ip-172-31-80-109:~$ sudo systemctl status jenkins
● jenkins.service - Jenkins Continuous Integration Server
     Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: enabled)
     Active: active (running) since Thu 2024-07-04 07:14:15 UTC; 2min 28s ago
   Main PID: 4252 (java)
      Tasks: 37 (limit: 1130)
     Memory: 310.8M (peak: 334.0M)
        CPU: 1min 2.865s
     CGroup: /system.slice/jenkins.service
             └─4252 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.war --webroot=/var/cache/jenkins/war --httpPort=8080

Jul 04 07:13:23 ip-172-31-80-109 jenkins[4252]: 12172b3563b449929bc1a98238d88317
Jul 04 07:13:23 ip-172-31-80-109 jenkins[4252]: This may also be found at: /var/lib/jenkins/secrets/initialAdminPassword
Jul 04 07:13:23 ip-172-31-80-109 jenkins[4252]: *************************************************************
Jul 04 07:13:23 ip-172-31-80-109 jenkins[4252]: *************************************************************
Jul 04 07:13:23 ip-172-31-80-109 jenkins[4252]: *************************************************************
Jul 04 07:14:15 ip-172-31-80-109 jenkins[4252]: 2024-07-04 07:14:15.465+0000 [id=29]	INFO	jenkins.InitReactorRunner$1#onAttained: Completed initialization
Jul 04 07:14:15 ip-172-31-80-109 jenkins[4252]: 2024-07-04 07:14:15.512+0000 [id=22]	INFO	hudson.lifecycle.Lifecycle#onReady: Jenkins is fully up and running
Jul 04 07:14:15 ip-172-31-80-109 systemd[1]: Started jenkins.service - Jenkins Continuous Integration Server.
Jul 04 07:14:15 ip-172-31-80-109 jenkins[4252]: 2024-07-04 07:14:15.650+0000 [id=45]	INFO	h.m.DownloadService$Downloadable$1load: Obtained the updated data file for hu
Jul 04 07:14:15 ip-172-31-80-109 jenkins[4252]: 2024-07-04 07:14:15.651+0000 [id=45]	INFO	hudson.util.Retrier#start: Performed the action check updates server success
lines 1-20/20 (END)
```

3.3 Now we need to add some **Inbound security group rules,** to do that open the EC2 Instance.



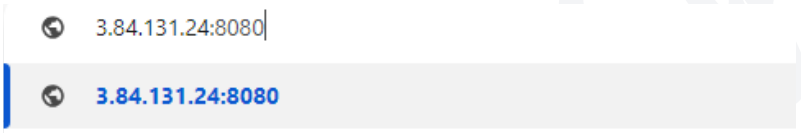
3.4 Now go to the **Security** section and Click on **Security group** then click on **Edit inbound rules,** add **Custom TCP rule** and port range is **8080** in source information Select from My IP and click on **Save rules.**

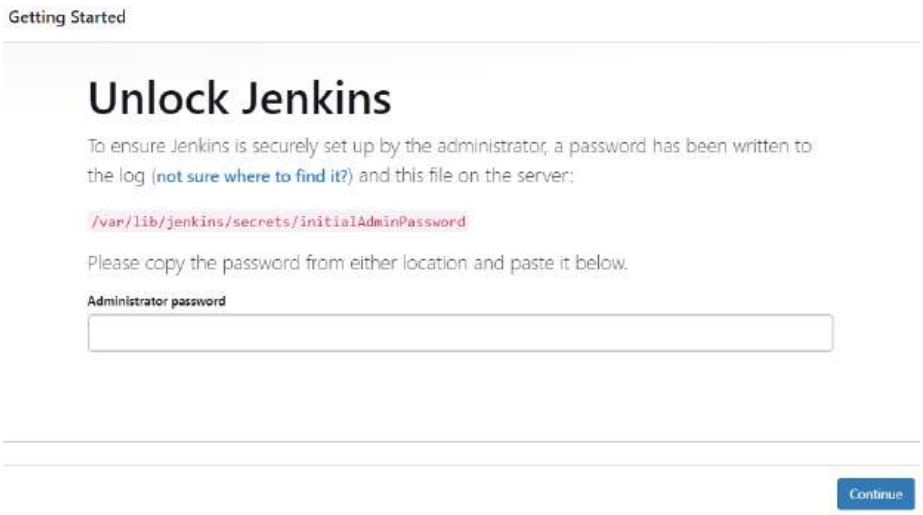




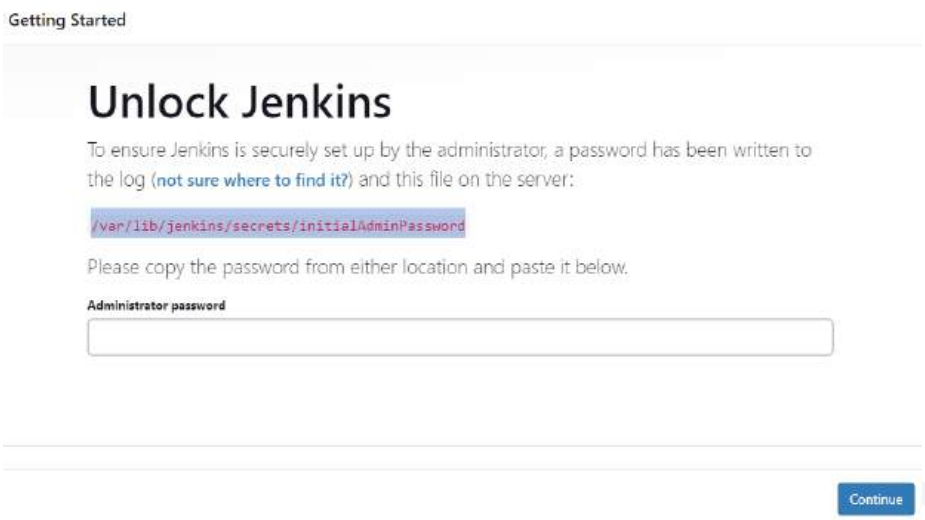
Step 4: Getting Started with Jenkins

4.1 Now Click on **Instance Public IP address** and paste it on **website URL,** and enter **port 8080** then click **enter,** then **jenkins will be started.**

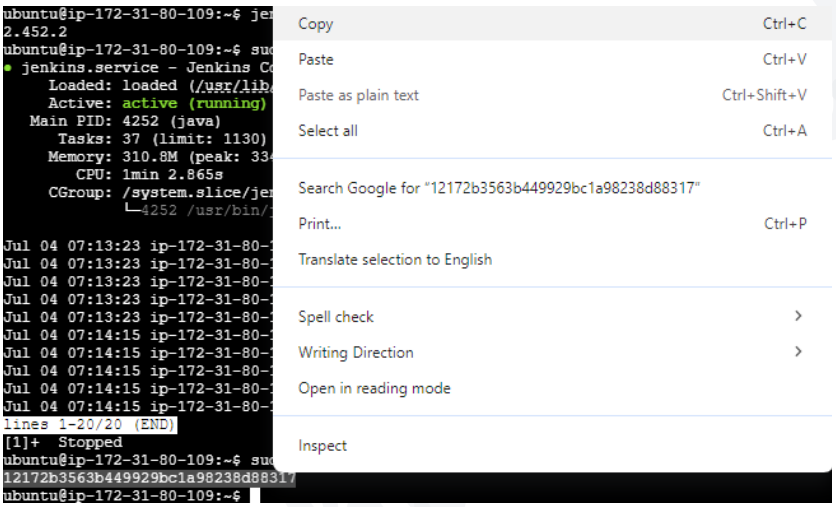


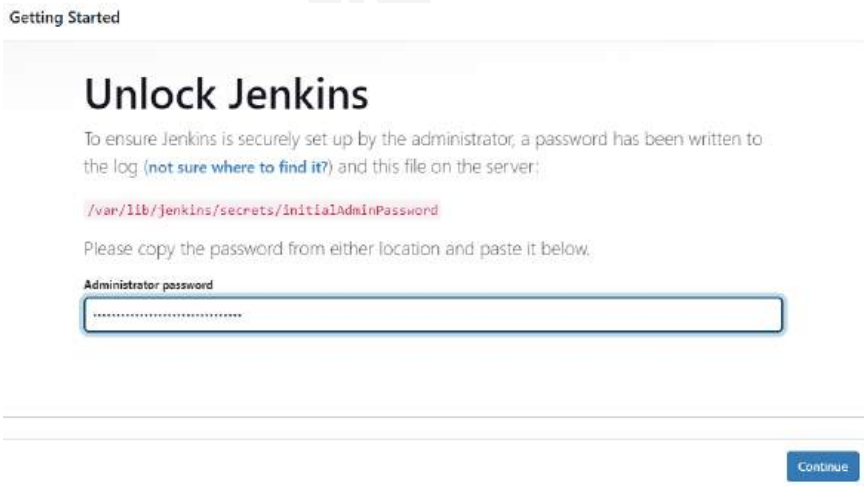


4.2 Now select **password path** and paste in on the **EC2 terminal** to get password



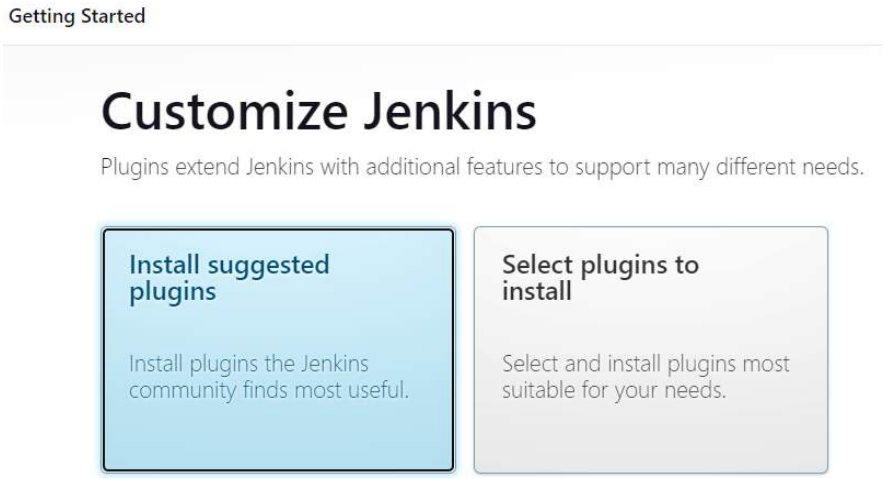

```
ubuntu@ip-172-31-80-109:~$ sudo cat /var/lib/jenkins/secrets/initialAdminPassword
12172b3563b449929bc1a98238d88317
ubuntu@ip-172-31-80-109:~$
```

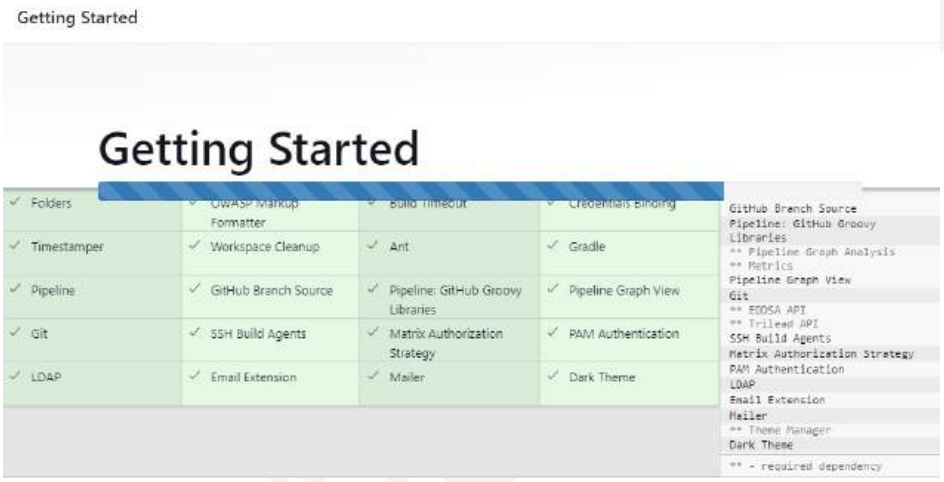


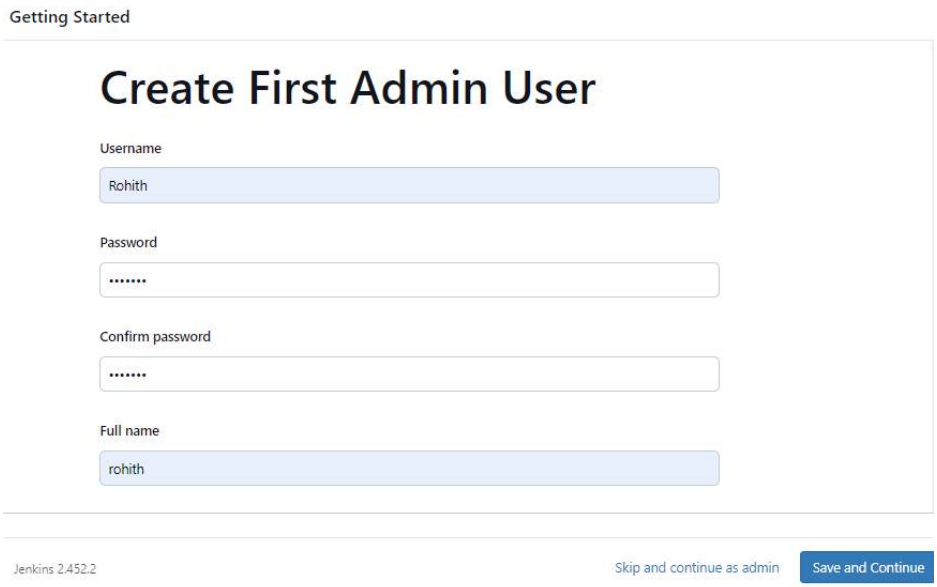


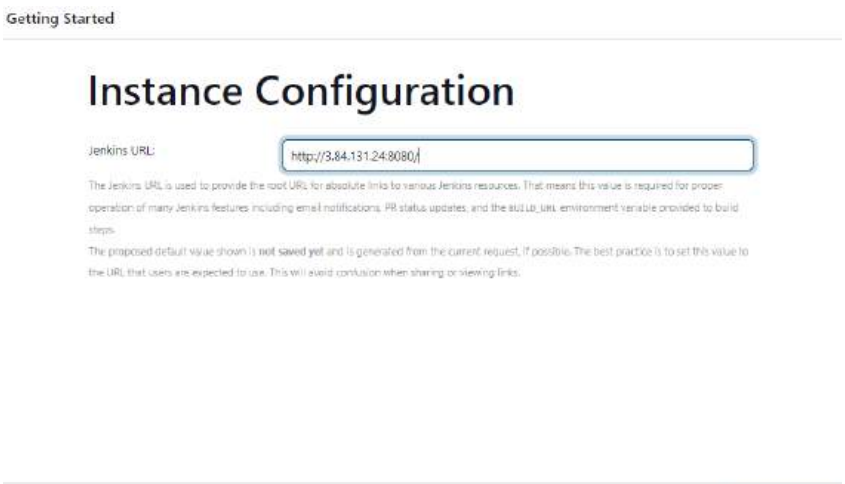
4.3 In **Customize Jenkins,** to install plugins, Select **Install suggested plugins,** then plugins downloading will **Getting started,** then Give **Username,**

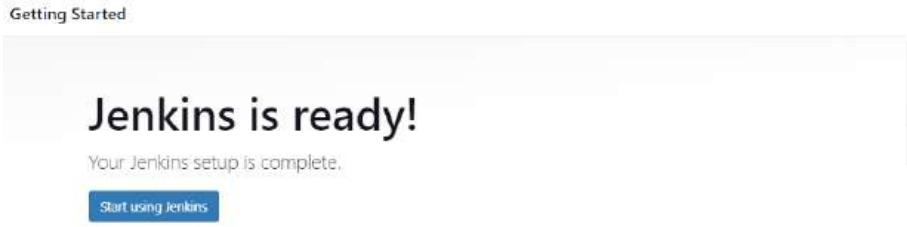
Password, Confirm password, full name and In **Instance Configuration** Give **jenkinsURL** and Click on **Save and Finish.**





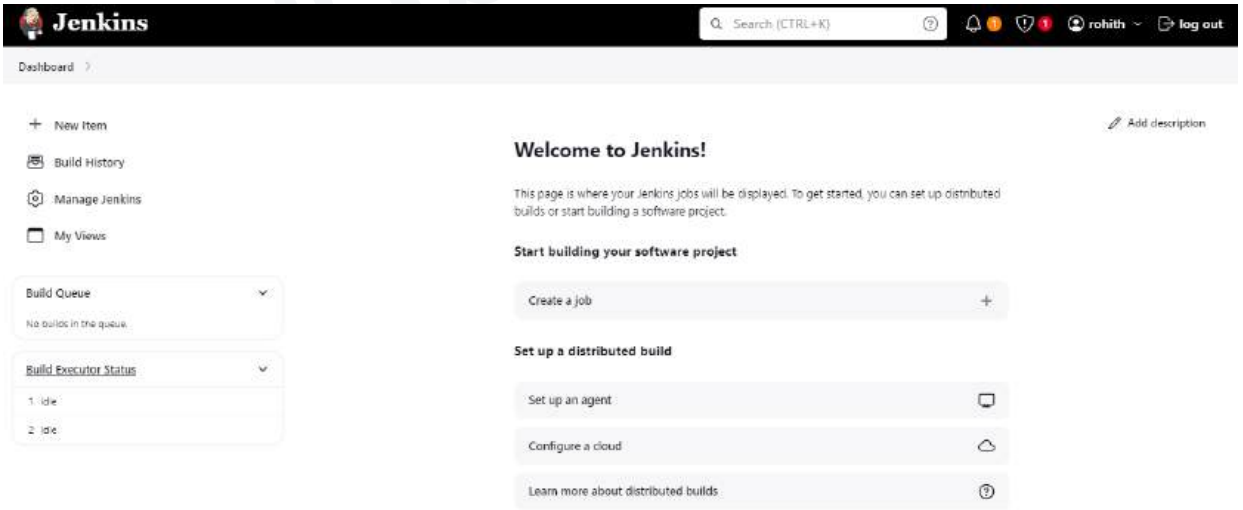


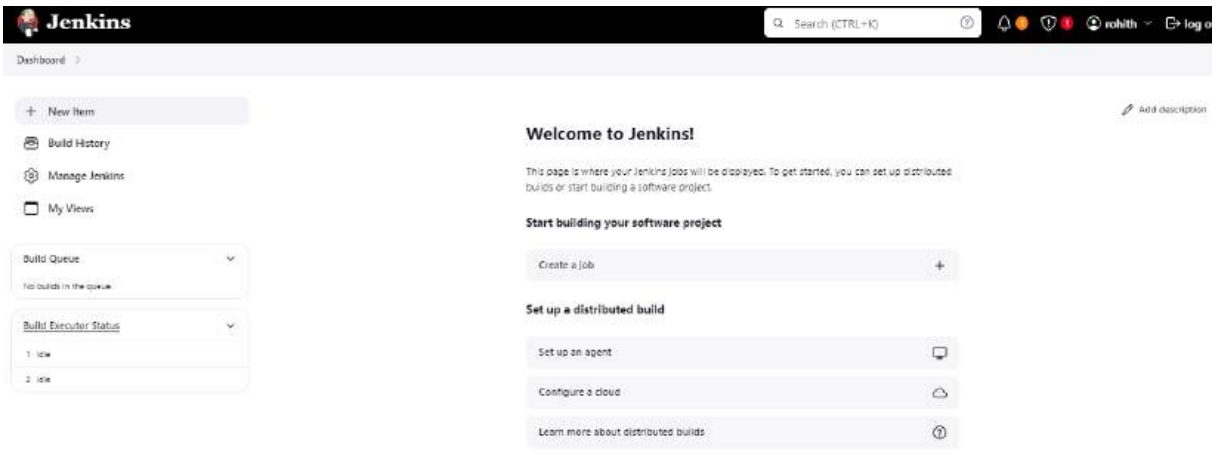


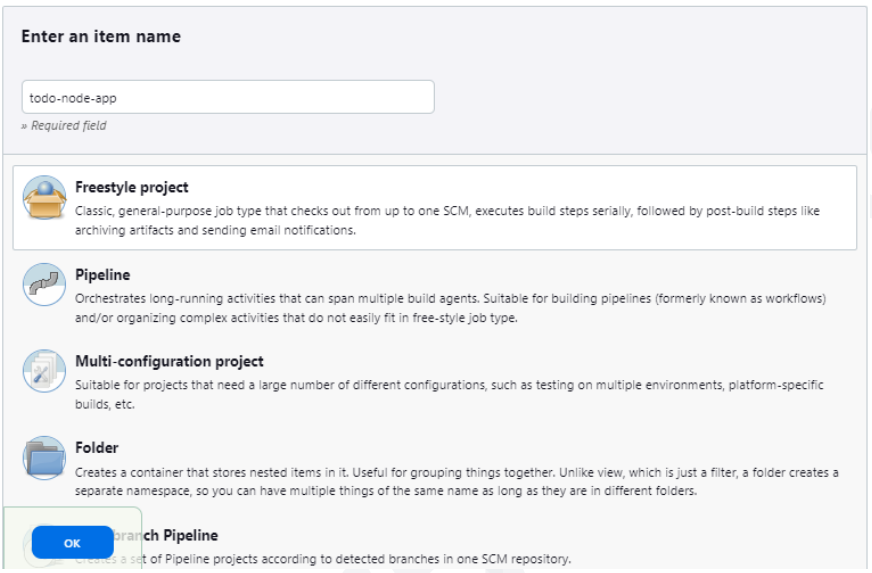


Step 5: Jenkins Integration with GitHub

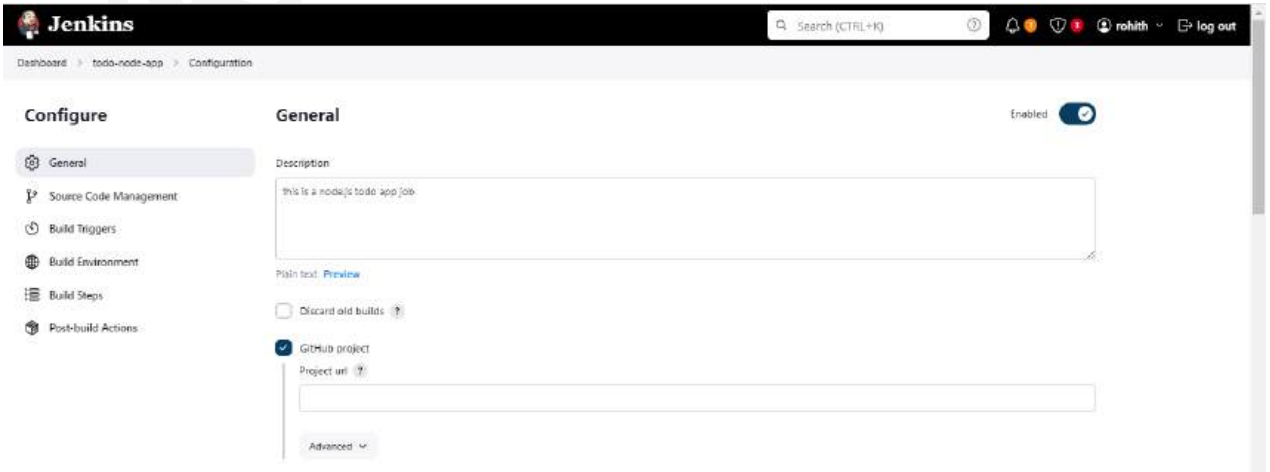
5.1 Now we will be entering into **Jenkins Dashboard,** now click on New item then to create a new job Click on **Create a job** now enter the job name in **Enter an item name,** then Click on **Freestyle project** and Click on **Ok.**

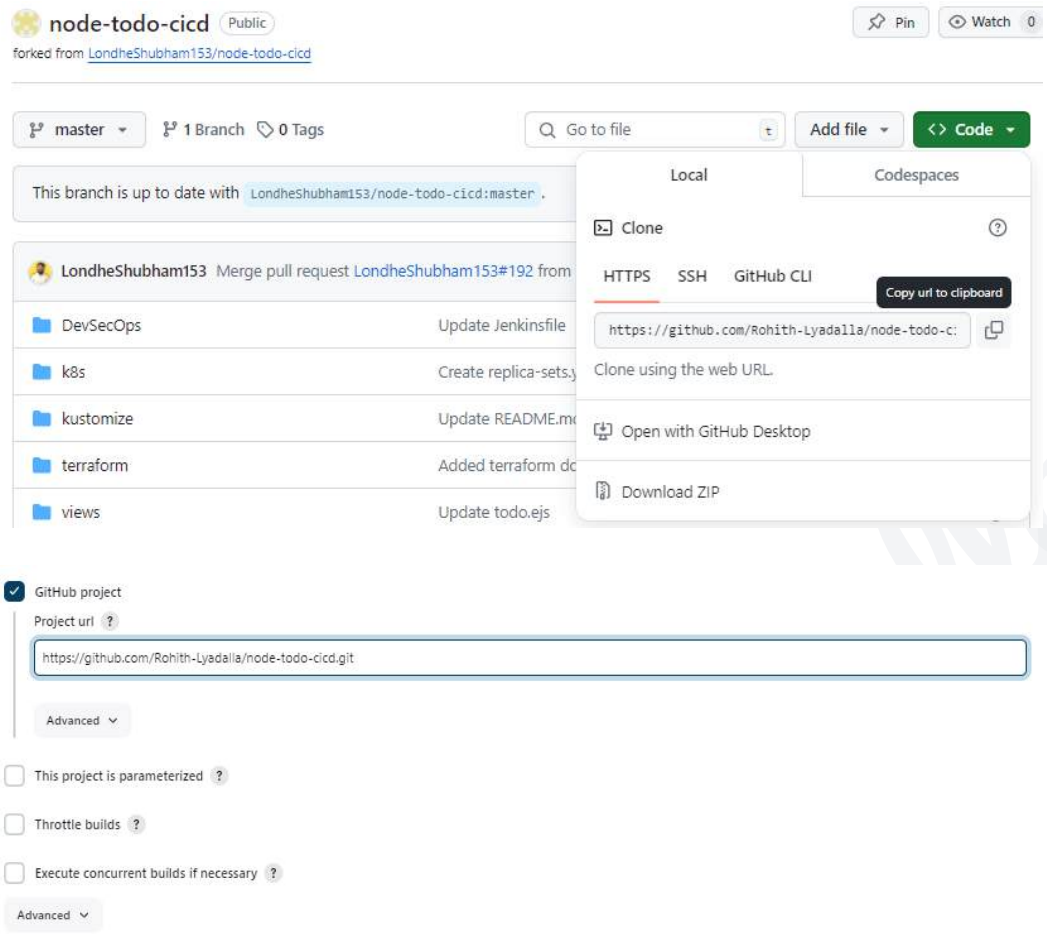






5.2 Now to **Configure** the project, In General give any **Description,** the **Check the GitHub project,**Now go to your **GitHub Repository** and now Click on **Code,** then **Copy the HTTPS URL** and now **paste** that **URL in Project URL in GitHub project.**





5.3 In **Source code management Check** the **Git,** then p**aste the GitHub Repository URL.**

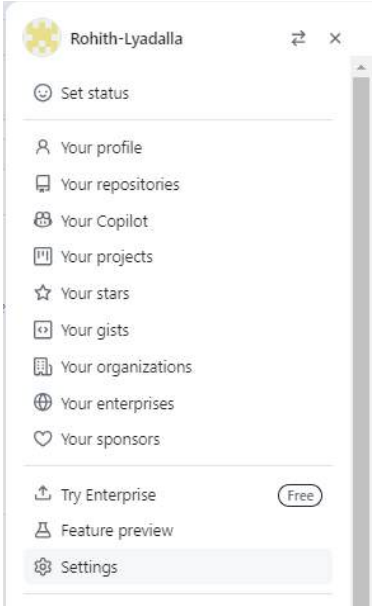
5.4 Now to **integrate jenkins with GitHub** we need to generate credentials, credentials are **Public key** and **private key,** to generate them **go to the terminal** and then Use the command **ssh -keygen,** then it will be generating public key and private key.

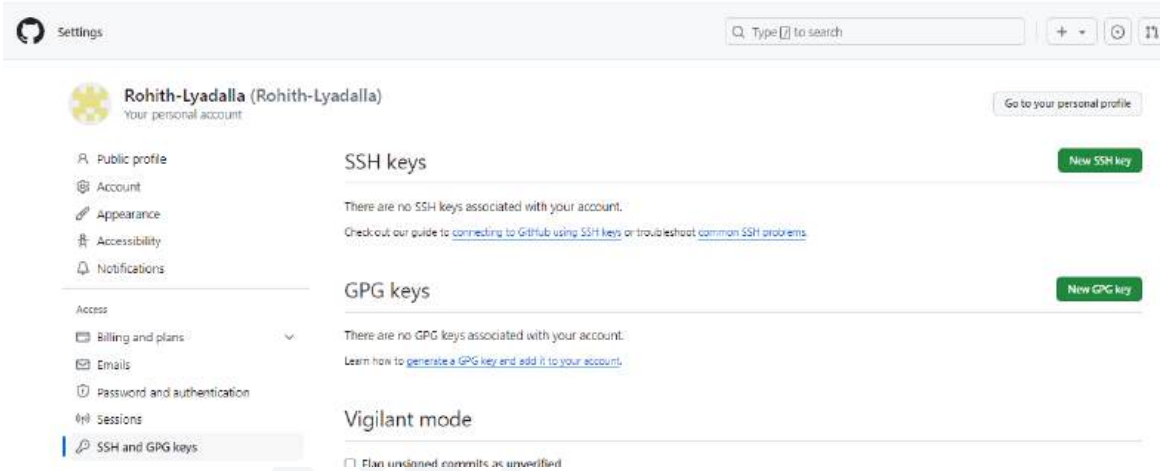
```
ubuntu@ip-172-31-80-109:~$ ssh-keygen
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/ubuntu/.ssh/id_ed25519):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/ubuntu/.ssh/id_ed25519
Your public key has been saved in /home/ubuntu/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:wuk76AC1SyljNQaG+cCgKGj/TzJ3aTlcLfiBxP8KcLg ubuntu@ip-172-31-80-109
The key's randomart image is:
+--[ED25519 256]--+
|=+               |
|O..     .        |
|=+.+     o       |
|o.=o.. .o + .    |
|+.+.  +oS+ = .   |
|.= ... .= = +    |
|  o  =.E O . .   |
|   .. B.o o .    |
|   .. .o   .     |
+----[SHA256]-----+
```

5.5 Now to **integrate jenkins with GitHub,** you need to use the public key by adding it on your github repository's SSH key. To do that you need to change the directory to ssh where keys are presented in it, for this use the command **cd .ssh** then list the keys using the command **ls,** so you will see the **public key** and **private key.**

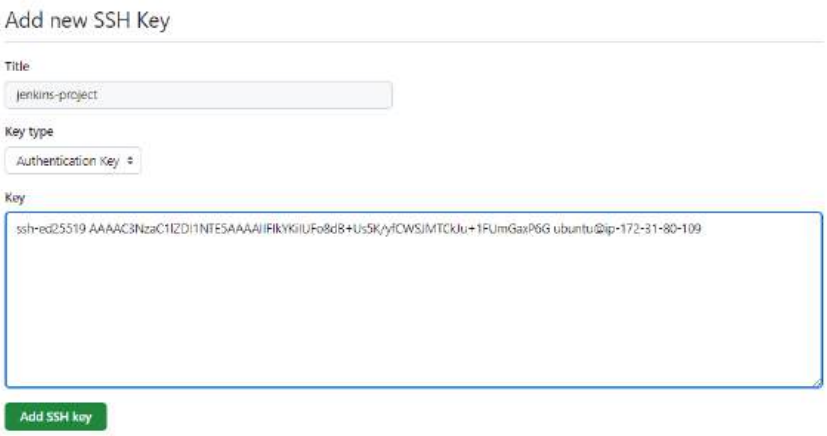
```
ubuntu@ip-172-31-80-109:~$ cd .ssh
ubuntu@ip-172-31-80-109:~/.ssh$ ls
authorized_keys  id_ed25519  id_ed25519.pub
ubuntu@ip-172-31-80-109:~/.ssh$ cat  id_ed25519
-----BEGIN OPENSSH PRIVATE KEY-----
b3BlbnNzaC1rZXktdjEAAAAABG5vbmUAAAAEbm9uZQAAAAAAAAABAAAAMwAAAAtzc2gtZW
QyNTUxOQAAACCBSJGCoiFBaPHQflLOSv8nwlkiTEwpCbvtRVJhmsT+hgAAAKBciu+qXIrv
qgAAAAtzc2gtZWQyNTUxOQAAACCBSJGCoiFBaPHQflLOSv8nwlkiTEwpCbvtRVJhmsT+hg
AAAEAuhfRQlhtiSYeRHrYvvO2a5cW9uzJmL6VdL4rmn1ndWYFIkYKiIUFo8dB+Us5K/yfC
WSJMTCkJu+1FUmGaxP6GAAAAF3VidW50dUBpcC0xNzItMzEtODAtMTA5AQIDBAUG
-----END OPENSSH PRIVATE KEY-----
ubuntu@ip-172-31-80-109:~/.ssh$ cat id_ed25519.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIIFIkYKiIUFo8dB+Us5K/yfCWSJMTCkJu+1FUmGaxP6G ubuntu@ip-172-31-80-109
ubuntu@ip-172-31-80-109:~/.ssh$
```

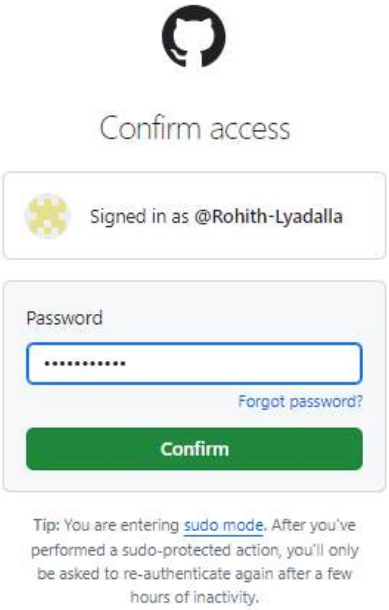
5.6 Now **go to the GitHub repository,** then click on your profile, then click on settings, now click on SSH and GPG keys, now Copy the public key and then paste it on SSH key and click on Add SSH key, now confirm the access with entering your GitHub password, now you can see that SSH keys are added.

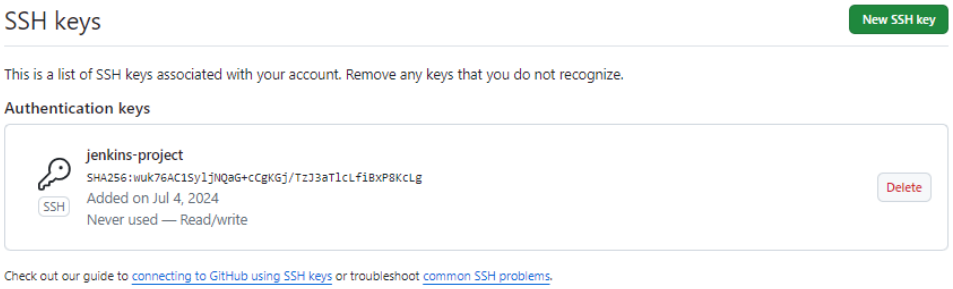




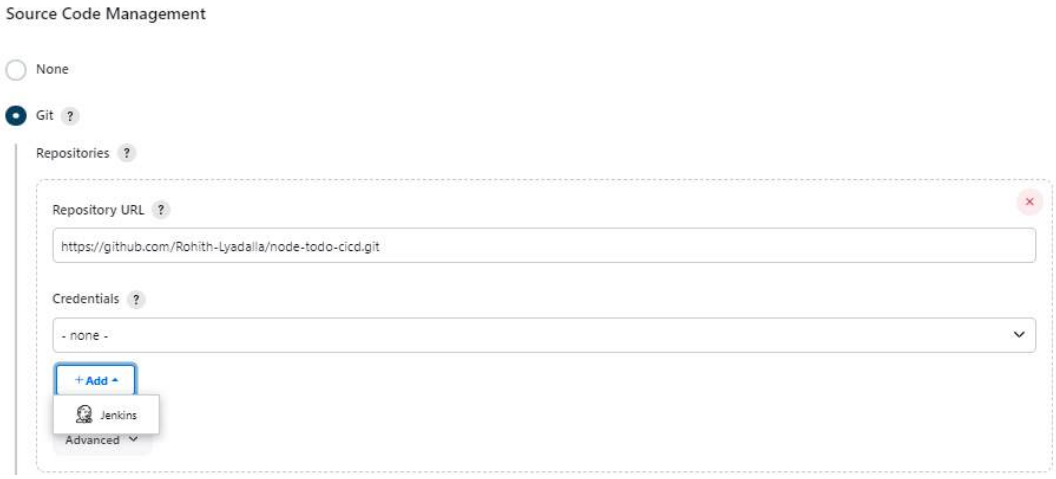

```
ubuntu@ip-172-31-80-109:~/.ssh$ cat id_ed25519.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIIFIkYKiIUFo8dB+Us5K/yfCWSJMTCkJu+1FUmGaxP6G ubuntu@ip-172-31-80-109
ubuntu@ip-172-31-80-109:~/.ssh$
```







5.7 In **Credentials,** Click on **add,** then **select Jenkins,** now **Jenkins Credentials Provider will be opening, select private key** and now **go to the terminal** and **select the private key** and **paste** it in the **private key in jenkins** and click on **Add,** then **save and apply.**



Jenkins Credentials Provider: Jenkins

Kind

SSH Username with private key

Scope ?

Global (Jenkins, nodes, items, all child items, etc)

ID ?

github-jenkins

Description ?

this is for jenkins and github integration

Username

ubuntu

Treat username as secret ?


```
-----BEGIN OPENSSH PRIVATE KEY-----
b3BlbnNzaC1rZXktdjEAAAAABG5vbmUAAAAEbm9uZQAAAAAAAAABAAAAMwAAAAtzc2gtZW
QyNTUxOQAAACCBSJGCoiFBaPHQflLOSv8nwlkiTEwpCbvtRVJhmsT+hgAAAKBciu+qXIrv
qgAAAAtzc2gtZWQyNTUxOQAAACCBSJGCoiFBaPHQflLOSv8nwlkiTEwpCbvtRVJhmsT+hg
AAAEAuhfRQlhtiSYeRHrYvvO2a5cW9uzJmL6VdL4rmn1ndWYFIkYKiIUFo8dB+Us5K/yfC
WSJMTCkJu+1FUmGaxP6GAAAAF3VidW50dUBpcC0xNzItMzEtODAtMTA5AQIDBAUG
-----END OPENSSH PRIVATE KEY-----
```


Private Key

Enter directly

Key

Enter New Secret Below

Passphrase

Cancel

Add

Git ?

Repositories ?

Repository URL ?

https://github.com/Rohith-Lyadalla/node-todo-cicd.git

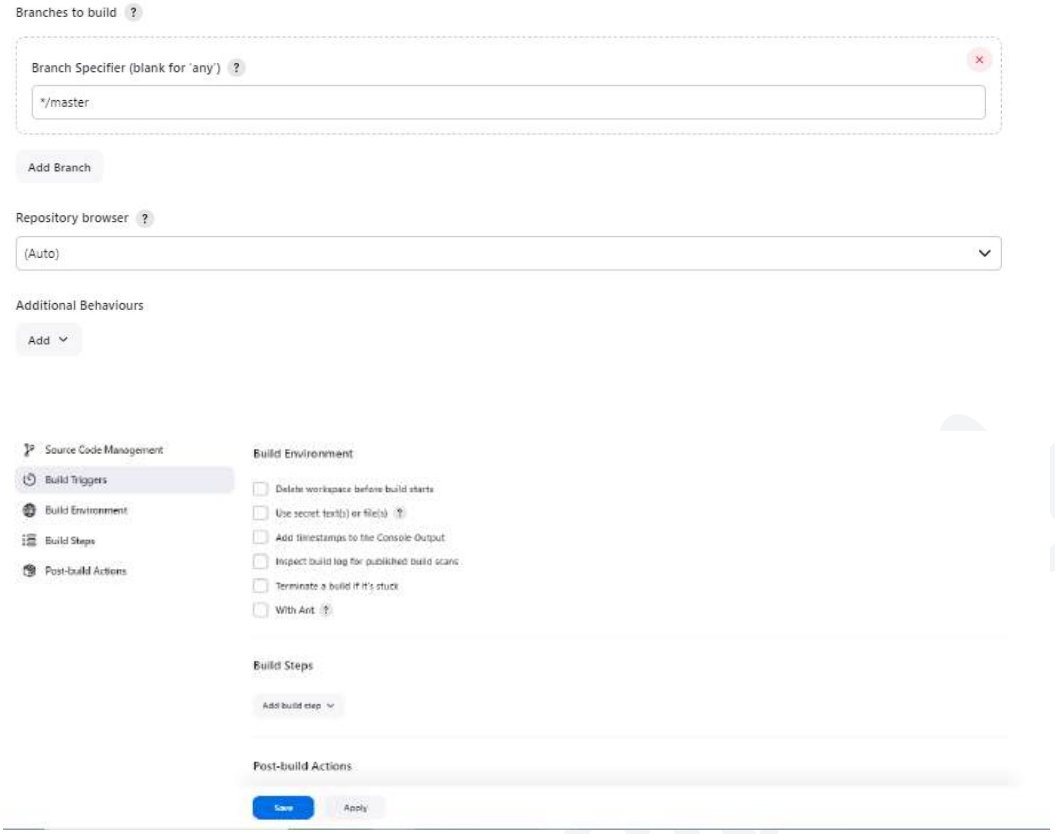
Credentials ?

ubuntu (this is for jenkins and github integration)

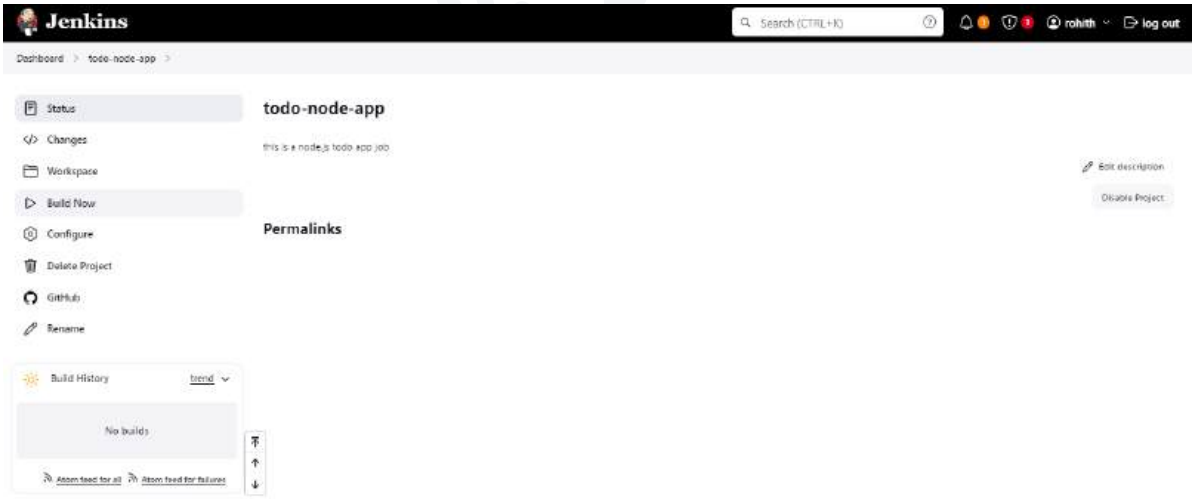
+ Add

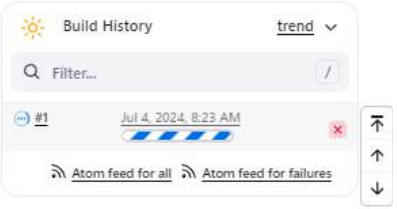
Advanced

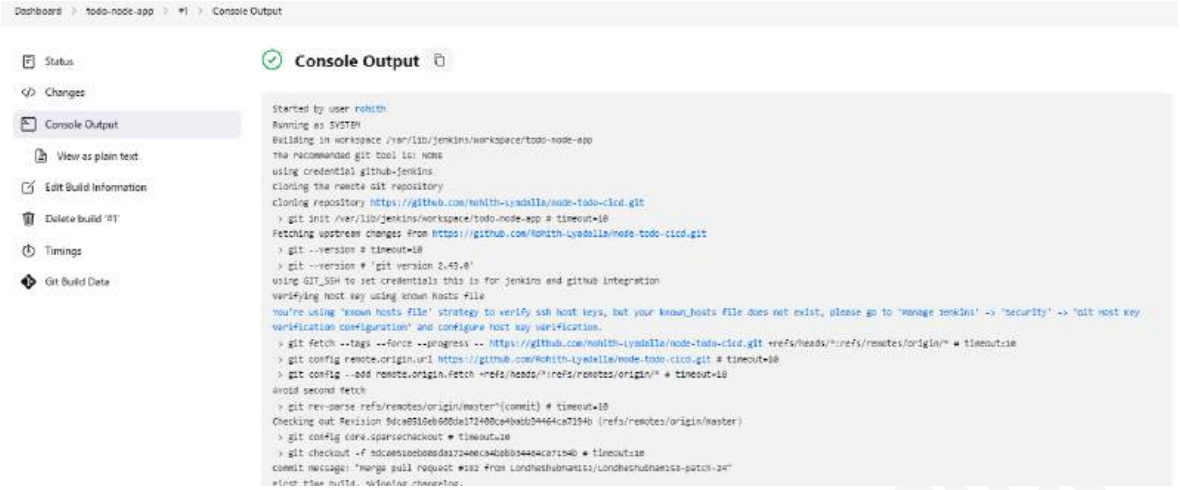
Add Repository



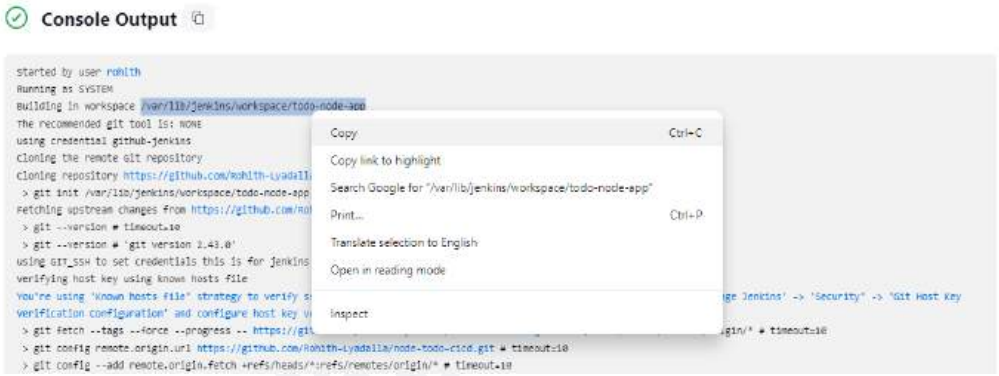
5.8 To Build the code of the application, click on Build Now, then code will be built.







5.9 Now **jenkins has been integrated with GitHub,** and the code has been stored in jenkins from GitHub, now to check what are the contents present in the jenkins to view use the command **cd** to change the directory to the directory where the code is stored and copy and paste path and now use the command **ls** to view all the contents.



5.10 Now we have code for application which is written in node.js, Now to run the Code which is written in node.js, Now go to Readme file use the following commands.

node-todo-cicd

Run these commands:

```
sudo apt install nodejs
```

```
sudo apt install npm
```

```
npm install
```

```
node app.js
```

```
ubuntu@ip-172-31-80-109:/var/lib/jenkins/workspace/todo-node-app$ sudo apt install nodejs
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  libcares2 libnode109 node-acorn node-busboy node-cjs-module-lexer node-undici node-xtend nodejs-doc
Suggested packages:
  npm
The following NEW packages will be installed:
  libcares2 libnode109 node-acorn node-busboy node-cjs-module-lexer node-undici node-xtend nodejs nodejs-doc
0 upgraded, 9 newly installed, 0 to remove and 0 not upgraded.
Need to get 16.1 MB of archives.
After this operation, 70.4 MB of additional disk space will be used.
Do you want to continue? [Y/n] y
```

```
Preparing to unpack .../7-libnode109_18.19.1+dfsg-6ubuntu5_amd64.deb ...
Unpacking libnode109:amd64 (18.19.1+dfsg-6ubuntu5) ...
Selecting previously unselected package nodejs-doc.
Preparing to unpack .../8-nodejs-doc_18.19.1+dfsg-6ubuntu5_all.deb ...
Unpacking nodejs-doc (18.19.1+dfsg-6ubuntu5) ...
Setting up node-cjs-module-lexer (1.2.3+dfsg-1) ...
Setting up libcares2:amd64 (1.27.0-1.0ubuntu1) ...
Setting up nodejs-doc (18.19.1+dfsg-6ubuntu5) ...
Setting up node-xtend (4.0.2-3) ...
Setting up node-busboy (1.6.04+~cs2.6.0-2) ...
Setting up node-undici (5.26.3+dfsg1+~cs23.10.12-2) ...
Setting up node-acorn (8.8.1+ds+~cs25.17.7-2) ...
Setting up libnode109:amd64 (18.19.1+dfsg-6ubuntu5) ...
Setting up nodejs (18.19.1+dfsg-6ubuntu5) ...
update-alternatives: using /usr/bin/nodejs to provide /usr/bin/js (js) in auto mode
Processing triggers for man-db (2.12.0-4build2) ...
Processing triggers for libc-bin (2.39-0ubuntu8.2) ...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@ip-172-31-80-109:/var/lib/jenkins/workspace/todo-node-app$
```

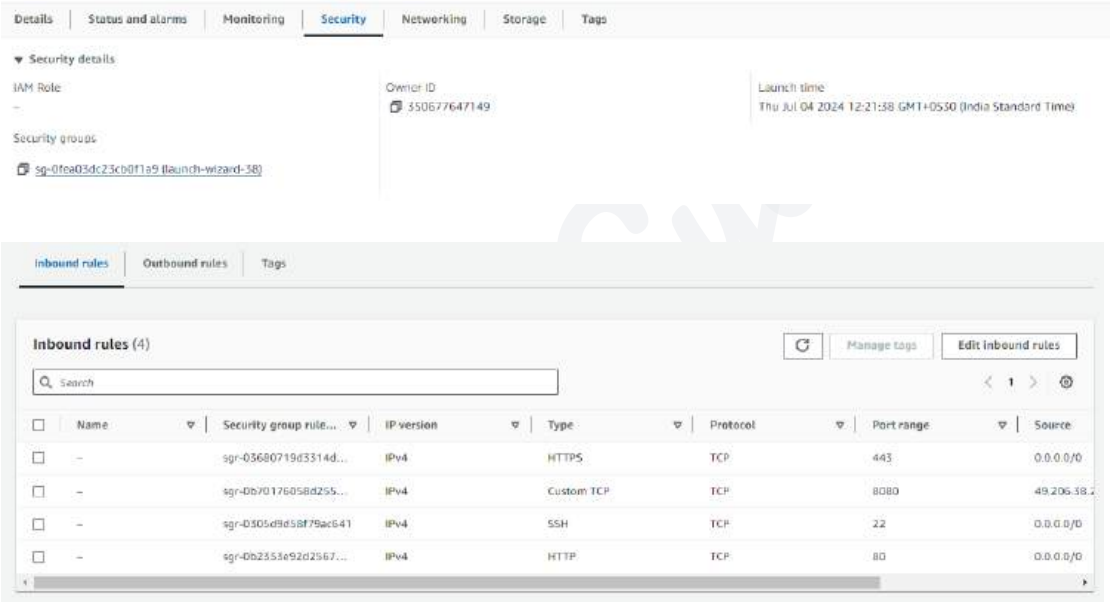
```
ubuntu@ip-172-31-80-109:/var/lib/jenkins/workspace/todo-node-app$ sudo apt install npm
```

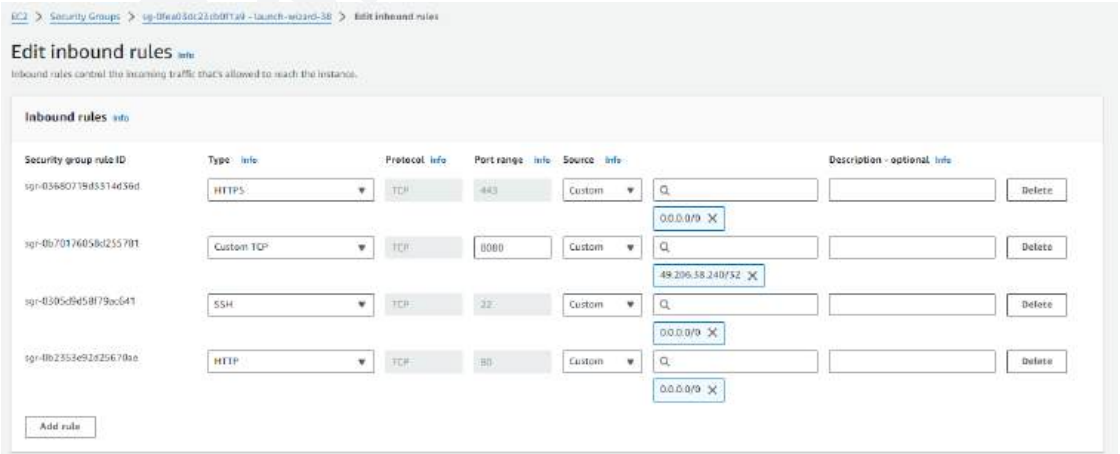
```
  node-foreground-child node-fs-readdir-recursive node-fs-write-stream-atomic node-fs.realpath node-function-bind node-functional-red-black-tree node-gauge node-get-caller-file
  node-get-stream node-get-value node-glob node-glob-parent node-globals node-globby node-got node-graceful-fs node-growl node-gyp node-has-flag node-has-unicode node-has-value
  node-has-values node-hosted-git-info node-http-proxy-agent node-https-proxy-agent node-iconv-lite node-icss-utils node-ieee754 node-iferr node-ignore node-imurmurhash
  node-indent-string node-inflight node-inherits node-ini node-interpret node-ip node-ip-regex node-is-arrayish node-is-binary-path node-is-buffer node-is-descriptor
  node-is-extendable node-is-extglob node-is-glob node-is-number node-is-path-cwd node-is-path-inside node-is-plain-obj node-is-plain-object node-is-primitive node-is-stream
  node-is-typedarray node-is-windows node-isarray node-isexe node-isobject node-istanbul node-jest-debbundle node-jest-worker node-js-tokens node-js-yaml node-jsesc node-json-buffer
  node-json-parse-better-errors node-json-schema node-json-schema-traverse node-json-stable-stringify node-jsonb node-jsonify node-jsonparse node-kind-of node-lcov-parse node-levn
  node-loader-runner node-locate-path node-lodash node-lodash-packages node-log-driver node-lowercase-keys node-lru-cache node-make-dir node-map-visit node-memfs node-memory-fs
  node-merge-stream node-micromatch node-mime node-mime-types node-mimic-response node-minimatch node-minimist node-minipass node-mkdirp node-move-concurrently
  node-ms node-mute-stream node-n3 node-negotiator node-neo-async node-npm node-normalize-package-data node-normalize-path node-npm-bundled node-npm-package-arg node-npm-run-path
  node-npmlog node-object-assign node-object-inspect node-object-visit node-once node-opener node-optimist node-optionator node-osenv node-p-cancelable node-p-limit node-p-locate
  node-p-map node-parse-json node-pascalcase node-path-dirname node-path-exists node-path-is-absolute node-path-is-inside node-path-type node-picocolors node-pify node-pkg-dir
  node-postcss node-postcss-modules-extract-imports node-postcss-modules-values node-postcss-value-parser node-prelude-ls node-process-nextick-args node-progress
  node-promise-inflight node-promise-retry node-promzard node-prr node-pump node-punycode node-quick-lru node-randombytes node-re2 node-read node-read-package-json node-read-pkg
  node-readable-stream node-readdirp node-rechoir node-regenerate node-regenerate-unicode-properties node-regenerator-runtime node-regenerator-transform node-regexp
  node-regexpu-core node-regjsgen node-regjsparser node-repeat-string node-require-directory node-require-from-string node-resolve node-resolve-cwd node-resolve-from
  node-restore-cursor node-resumer node-retry node-rimraf node-run-queue node-safe-buffer node-schema-utils node-sellside-emitter node-semver node-serialize-javascript
  node-set-blocking node-set-immediate-shim node-set-value node-shebang-command node-shebang-regex node-shell-quote node-signal-exit node-slash node-slice-ansi node-source-list-map
  node-source-map node-source-map-support node-spdx-correct node-spdx-exceptions node-spdx-expression-parse node-spdx-license-ids node-sprintf-js node-ssri node-stack-utils
  node-string-decoder node-string-width node-strip-ansi node-strip-bom node-strip-json-comments node-supports-color node-tap node-tap-mocha-reporter node-tap-parser node-tapable
  node-tape node-tar node-terser node-text-table node-through node-time-stamp node-to-fast-properties node-to-regex-range node-tslib node-type-check node-typedarray
  node-typedarray-to-buffer node-unicode-canonical-property-names-ecmascript node-unicode-match-property-ecmascript node-unicode-match-property-value-ecmascript
  node-unicode-property-aliases-ecmascript node-unix-value node-unique-filename node-unset-value node-uri-js node-util node-util-deprecate node-uuid node-v8-compile-cache
  node-v8flags node-validate-npm-package-license node-validate-npm-package-name node-watchpack node-wcwidth.js node-webassemblyjs node-webpack-sources node-which node-wide-align
  node-widest-line node-wordwrap node-wrap-ansi node-wrappy node-write node-write-file-atomic node-ws node-y18n node-yallist node-yaml node-yargs node-yargs-parser npm
  perl-openssl-defaults xpcsvc-proto terser webpack x11-xserver-utils xdg-utils zutty
0 upgraded, 511 newly installed, 0 to remove and 0 not upgraded.
Need to get 95.5 MB of archives.
After this operation, 405 MB of additional disk space will be used.
Do you want to continue? [Y/n] y
```

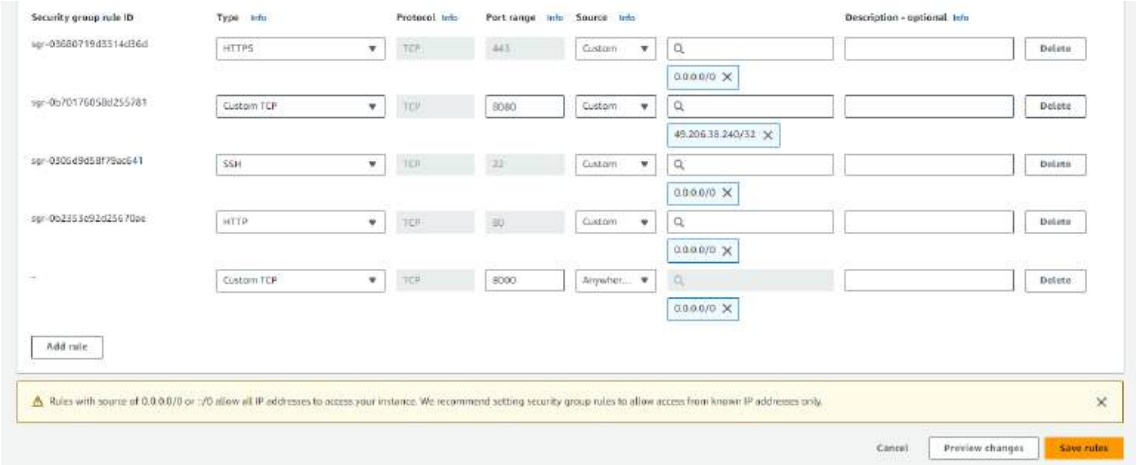
```
Setting up libxml-twig-perl (1:3.52-2) ...
Setting up libnet-dbus-perl (1.2.0-2build3) ...
Setting up node-util (0.12.5+~1.0.10-1) ...
Setting up node-babel-plugin-lodash (3.3.4+~cs2.0.1-7) ...
Setting up node-jest-debbundle (29.6.2-ds1+~cs73.45.28-5) ...
Setting up node-assert (2.0.0+~cs3.9.8-2) ...
Setting up webpack (5.76.1+dfsg1+~cs17.16.16-1) ...
Setting up node-parse-json (5.2.0+~cs5.1.7-1) ...
Setting up node-read-pkg (5.2.0-2) ...
Setting up node-istanbul (0.4.5+repack10+~cs98.25.59-2) ...
Setting up node-tape (5.6.1+~cs8.20.19-1) ...
Setting up node-css-loader (6.8.1+~cs14.0.17-1) ...
Setting up node-tap (16.3.7+ds1+~cs50.9.19-4) ...
Setting up node-deep-equal (2.2.3+~cs43.15.94-1) ...
Setting up npm (9.2.0~ds1-2) ...
Processing triggers for libc-bin (2.39-0ubuntu8.2) ...
Processing triggers for man-db (2.12.0-4build2) ...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@ip-172-31-80-109:/var/lib/jenkins/workspace/todo-node-app$
```

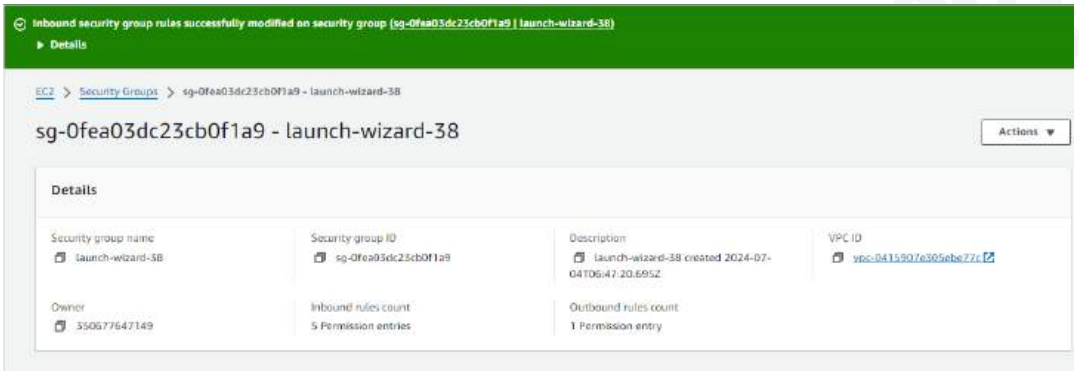
```
ubuntu@ip-172-31-80-109:/var/lib/jenkins/workspace/todo-node-app$ ls
DevSecOps  Dockerfile  Jenkinsfile  README.md  app.js  docker-compose.yaml  k8s  kustomize  package-lock.json  package.json  sonar-project.properties  terraform  test.js  views
ubuntu@ip-172-31-80-109:/var/lib/jenkins/workspace/todo-node-app$ sudo npm install
```

5.11 Now the **application To-do-list** is running on **port number 8000,** Here **we don't have access to port 8000 in the security group,** to give access, Now go to **EC2 Instance, select the instance,** go to **security** and Click on **Security group,** now add TCP inbound rule and click on save rules**.**

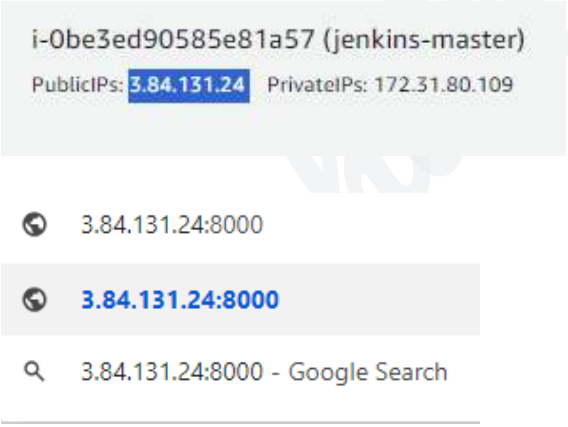


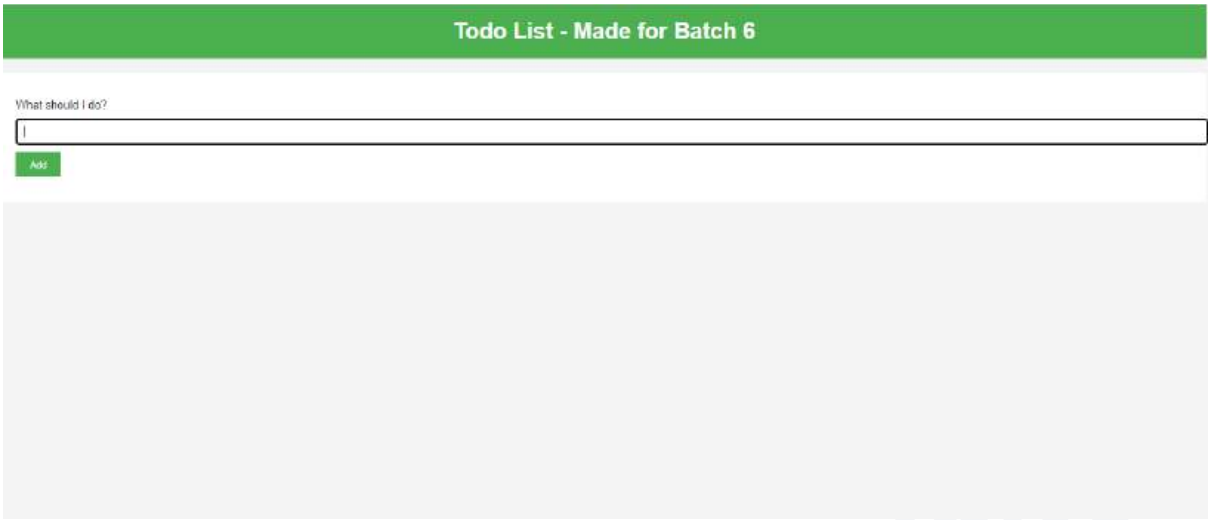




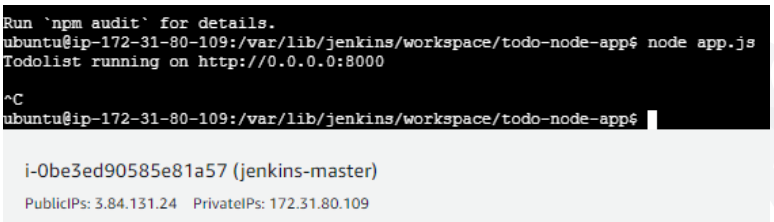


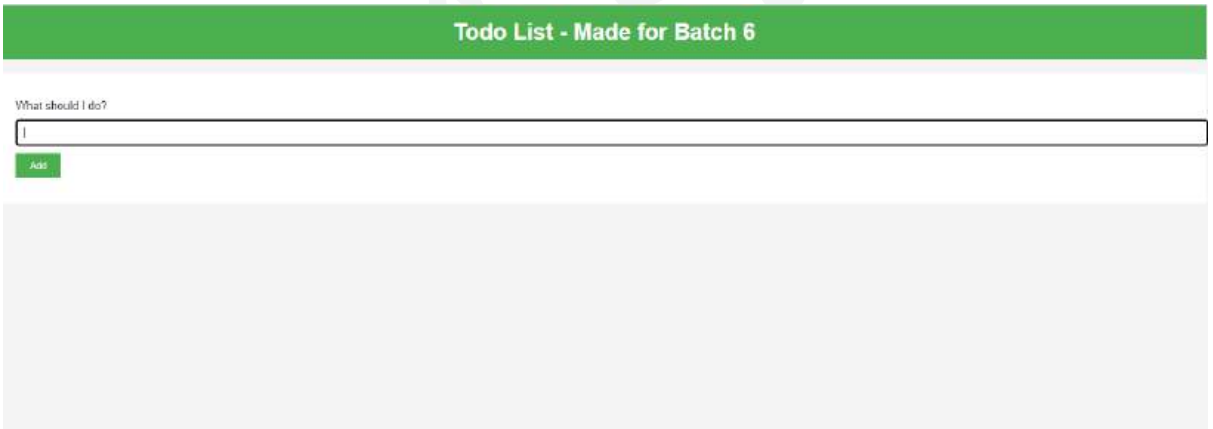
5.12 Now to access the application, copy the EC2 Instance Public ID and now paste it on the web browser's URL and enter the port number.

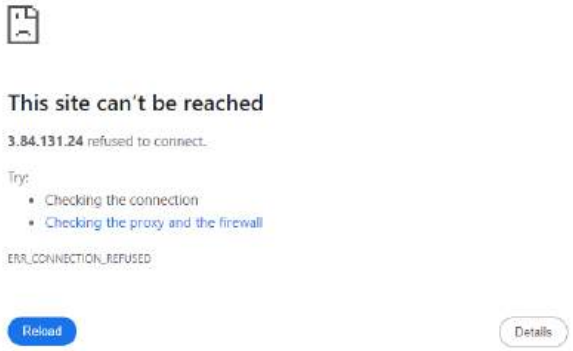




5.13 To stop the running application on the web browser, go to the terminal and enter Ctrl+C and now refresh the page, now you can see the application will be stopped running.







Step 6: Running the application in containerization environment

6.1 To run the application in the containerization environment we use Docker, here dockerfile is already presented, now we need to remove the existing Dockerfile and we need to run new dockerfile, to do that, use the commands.

```
ubuntu@ip-172-31-80-109:/var/lib/jenkins/workspace/todo-node-app$ ls
DevSecOps   Jenkinsfile  app.js              k8s        node_modules       package.json              terraform  views
Dockerfile  README.md    docker-compose.yaml kustomize  package-lock.json  sonar-project.properties  test.js
ubuntu@ip-172-31-80-109:/var/lib/jenkins/workspace/todo-node-app$
```

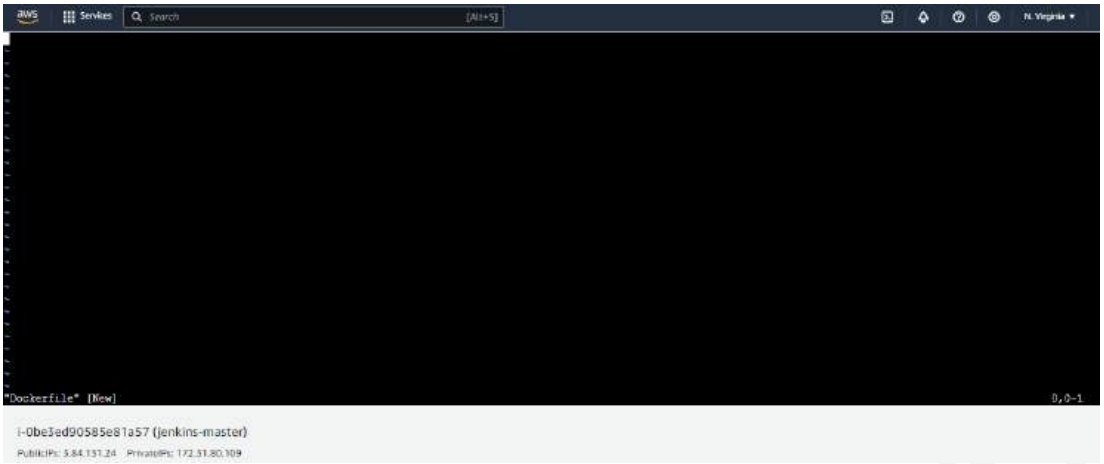
```
ubuntu@ip-172-31-80-109:/var/lib/jenkins/workspace/todo-node-app$ sudo rm Dockerfile
ubuntu@ip-172-31-80-109:/var/lib/jenkins/workspace/todo-node-app$ ls
DevSecOps  Jenkinsfile  README.md  app.js  docker-compose.yaml  k8s  kustomize  node_modules  package-lock.json  package.json  sonar-project.properties  terraform  test.js  views
ubuntu@ip-172-31-80-109:/var/lib/jenkins/workspace/todo-node-app$
```

6.2 Now we have to install Docker, to install the Docker follow the commands.

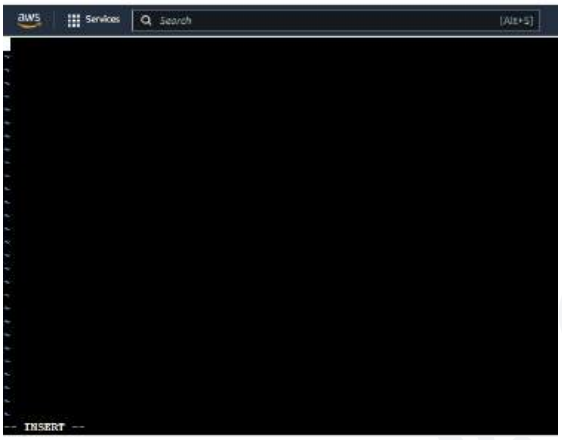
```
ubuntu@ip-172-31-80-109:/var/lib/jenkins/workspace/todo-node-app$ sudo apt install docker.io
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  bridge-utils containerd dns-root-data dnsmasq-base pigz runc ubuntu-fan
Suggested packages:
  ifupdown aufs-tools cgroupfs-mount | cgroup-lite debootstrap docker-buildx docker-compose-v2 docker-doc rinse zfs-fuse | zfsutils
The following NEW packages will be installed:
  bridge-utils containerd dns-root-data dnsmasq-base docker.io pigz runc ubuntu-fan
0 upgraded, 8 newly installed, 0 to remove and 0 not upgraded.
Need to get 76.8 MB of archives.
After this operation, 289 MB of additional disk space will be used.
Do you want to continue? [Y/n] y
```

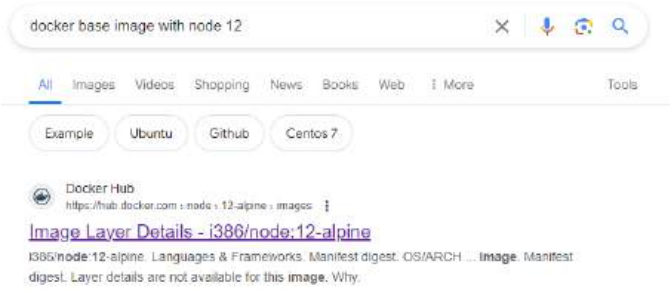
6.3 To create a Dockerfile use the command **sudo vim dockerfile**

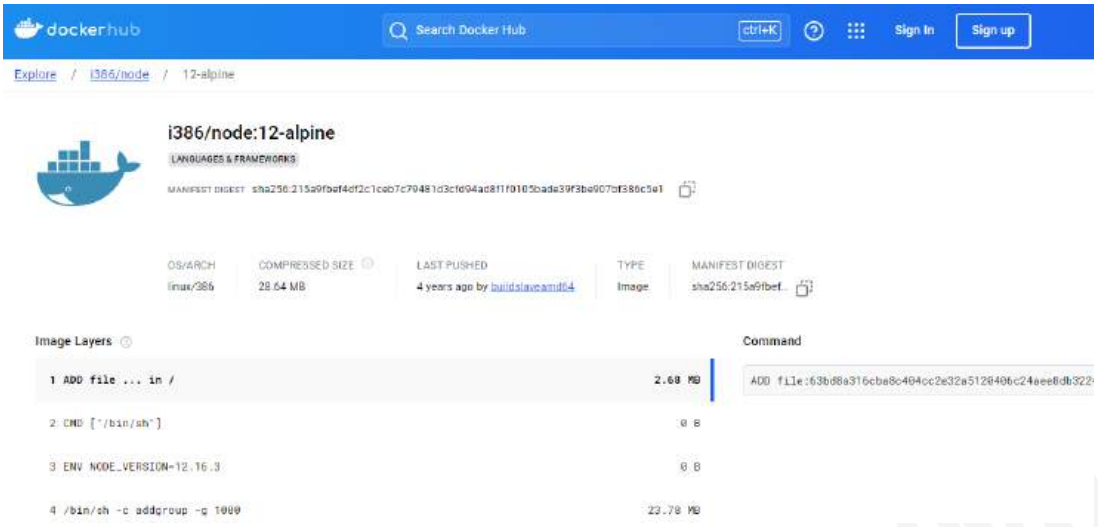
```
ubuntu@ip-172-31-80-109:/var/lib/jenkins/workspace/todo-node-app$ sudo vim Dockerfile
```

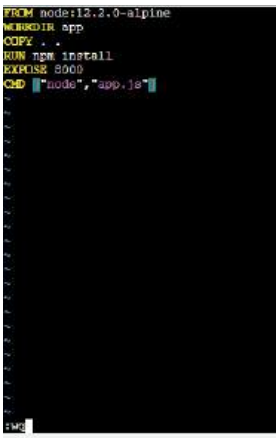


Enter **I** to insert









To check the Dockerfile is present on the virtual environment, use the command **ls**

```
ubuntu@ip-172-31-80-109:/var/lib/jenkins/workspace/todo-node-app$ ls
DevSecOps   Jenkinsfile  app.js              k8s        node_modules       package.json             terraform  views
Dockerfile  README.md    docker-compose.yaml kustomize  package-lock.json  sonar-project.properties test.js
ubuntu@ip-172-31-80-109:/var/lib/jenkins/workspace/todo-node-app$
```

6.4 Now to build the docker from dockerfile use the following command

```
ubuntu@ip-172-31-80-109:/var/lib/jenkins/workspace/todo-node-app$ sudo docker build -t todo-node-app .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  25.34MB
Step 1/6 : FROM node:12.2.0-alpine
12.2.0-alpine: Pulling from library/node
e7c96db7181b: Pull complete
a9b145f64bbe: Pull complete
3bcb5e14be53: Pull complete
Digest: sha256:2ab3d9a1bac67c9b4202b774664adaa94d2f1e426d8d28e07bf8979df61c8694
Status: Downloaded newer image for node:12.2.0-alpine
```

```
Step 3/6 : COPY . .
 ---> 3cbbb7852fa5
Step 4/6 : RUN npm install
 ---> Running in 6ff5667305ce
npm WARN read-shrinkwrap This version of npm is compatible with lockfileVersion@1, but package-lock.json was generated for lockfileVersion@2. I'll try to do my best with it!

> ejs@2.7.4 postinstall /app/node_modules/ejs
> node ./postinstall.js

Thank you for installing EJS: built with the Jake JavaScript build tool (https://jakejs.com/)

npm WARN my-todolist@0.1.0 No repository field.
npm WARN my-todolist@0.1.0 No license field.

updated 291 packages and audited 291 packages in 10.386s
found 17 vulnerabilities (1 low, 7 moderate, 6 high, 3 critical)
  run `npm audit fix` to fix them, or `npm audit` for details
Removing intermediate container 6ff5667305ce
 ---> 331a8fab3cb9
Step 5/6 : EXPOSE 8000
 ---> Running in 900ccb9d60bc
Removing intermediate container 900ccb9d60bc
 ---> f7e1f4f8a059
Step 6/6 : CMD ["node","app.js"]
 ---> Running in 6457a1185108
Removing intermediate container 6457a1185108
 ---> d0fb2c145b35
Successfully built d0fb2c145b35
Successfully tagged todo-node-app:latest
ubuntu@ip-172-31-80-109:/var/lib/jenkins/workspace/todo-node-app$
```

6.5 To check the contents in Docker, use the following command.

```
 ---> d0fb2c145b35
Successfully built d0fb2c145b35
Successfully tagged todo-node-app:latest
ubuntu@ip-172-31-80-109:/var/lib/jenkins/workspace/todo-node-app$ docker images
permission denied while trying to connect to the Docker daemon socket at unix:///var/run/docker.sock: Get "http://%2Fvar%2Frun%2Fdocker.sock/v1.24/images/json": dial unix /var/run/d
ker.sock: connect: permission denied
ubuntu@ip-172-31-80-109:/var/lib/jenkins/workspace/todo-node-app$ sudo docker images
REPOSITORY    TAG          IMAGE ID       CREATED          SIZE
todo-node-app  latest      d0fb2c145b35   4 minutes ago    124MB
node          12.2.0-alpine  f391dabf9dce   5 years ago      77.7MB
ubuntu@ip-172-31-80-109:/var/lib/jenkins/workspace/todo-node-app$
```

6.6 Now to run the container use the following command.

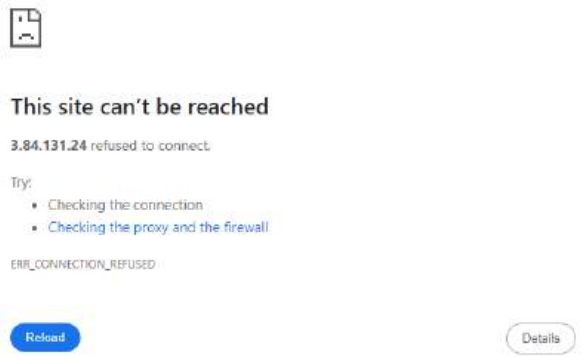
```
ubuntu@ip-172-31-80-109:/var/lib/jenkins/workspace/todo-node-app$ sudo docker run -d --name node-todo-app -p 8000:8000 todo-node-app
a458cfa4d7a0c3d9a55cdd867e121ea8cdf252615f4039fcef18a4eec71c9587
ubuntu@ip-172-31-80-109:/var/lib/jenkins/workspace/todo-node-app$
```

6.7 Now to check the state of the docker image running or not use the following command.

```
ubuntu@ip-172-31-80-109:/var/lib/jenkins/workspace/todo-node-app$ sudo docker ps
CONTAINER ID   IMAGE          COMMAND          CREATED          STATUS          PORTS                                       NAMES
a458cfa4d7a0   todo-node-app  "node app.js"    3 minutes ago    Up 3 minutes    0.0.0.0:8000->8000/tcp, :::8000->8000/tcp   node-todo-app
ubuntu@ip-172-31-80-109:/var/lib/jenkins/workspace/todo-node-app$
```

The above command is showing the container which is running.

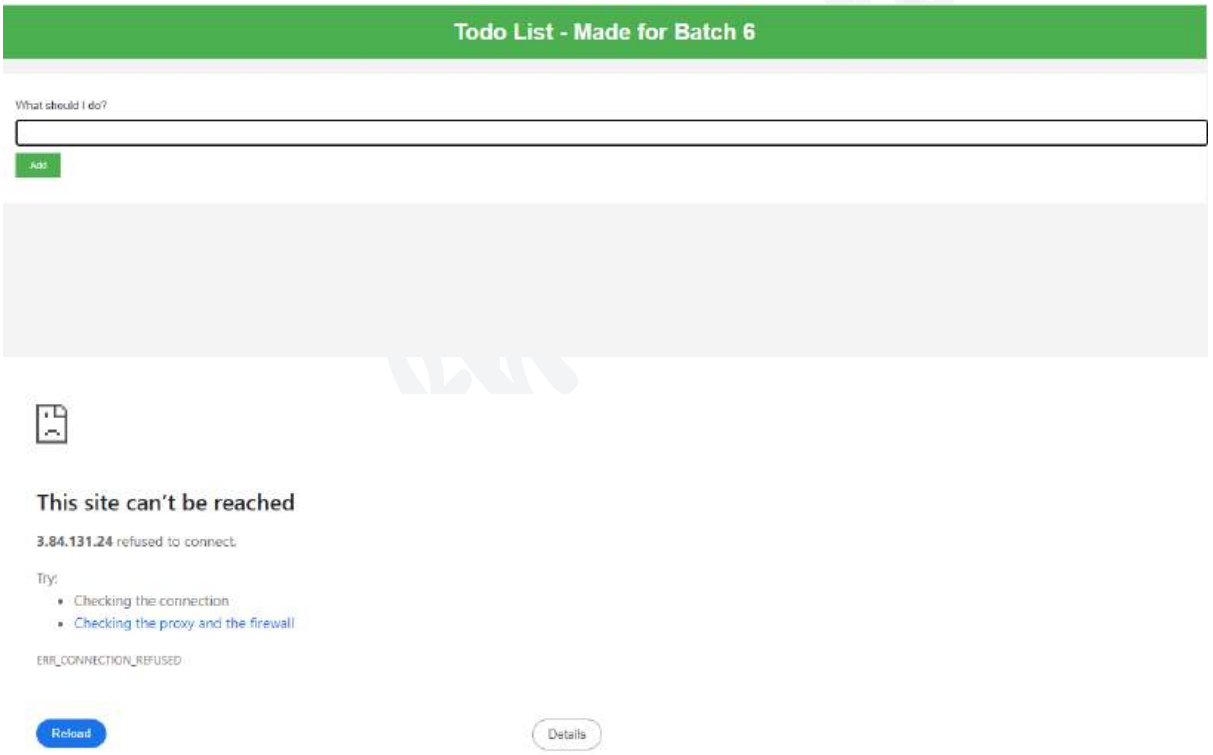
6.8 to check the application is running through Docker, go to web browser and refresh the page.



6.9 Now to delete the image Use the following command

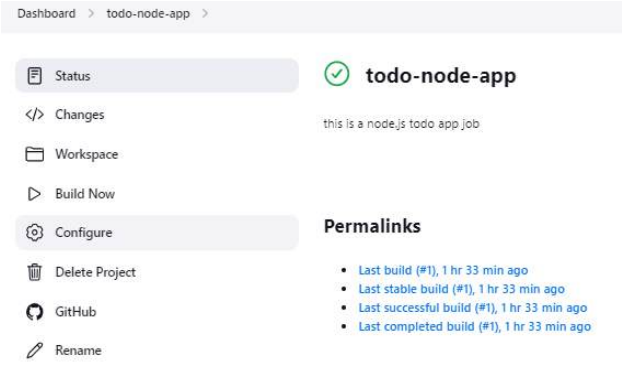
```
ubuntu@ip-172-31-80-109:/var/lib/jenkins/workspace/todo-node-app$ sudo docker kill a458cfa4d7a0
a458cfa4d7a0
ubuntu@ip-172-31-80-109:/var/lib/jenkins/workspace/todo-node-app$
```

6.10 Now to check if the container is deleted, go to the web browser and refresh the page.

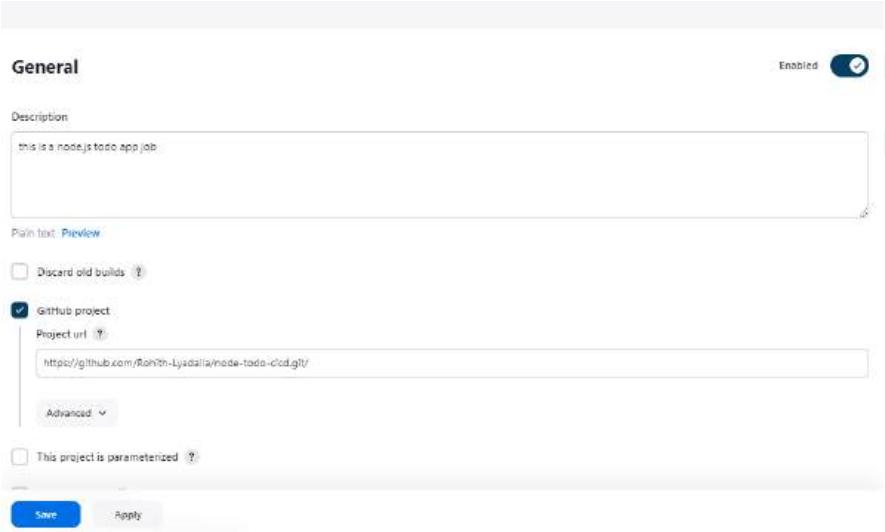


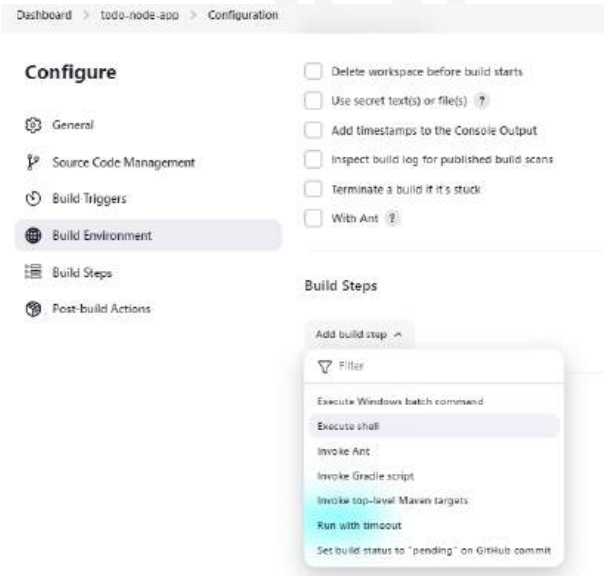
Step 7: Automation of CI/CD Pipeline

7.1 Till now whatever we have done manually, Now we will automate the things, to do that Click on **Configure.**

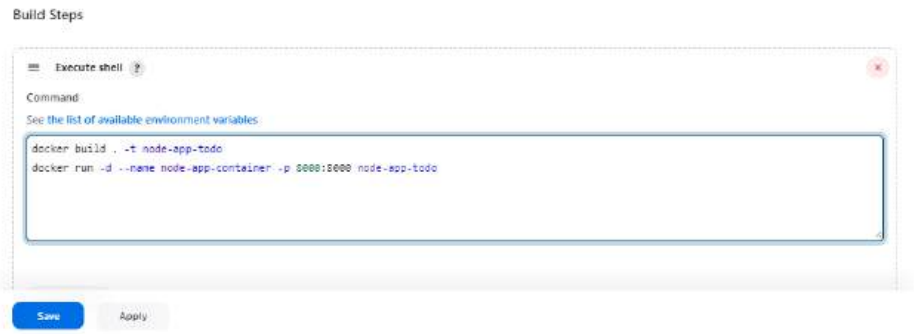


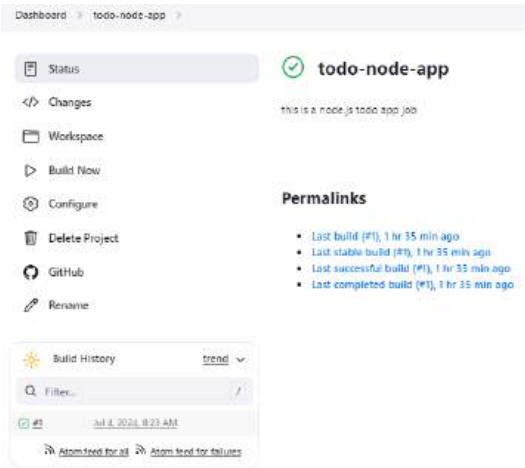
7.2 Now you will be entering into the Configuration section, Scroll Down to Build Environment, In build steps, Add build step, select Execute shell, In command enter docker build and docker run, then click on save.



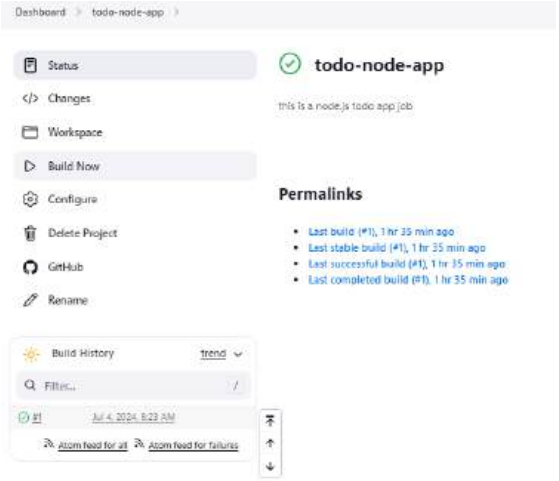


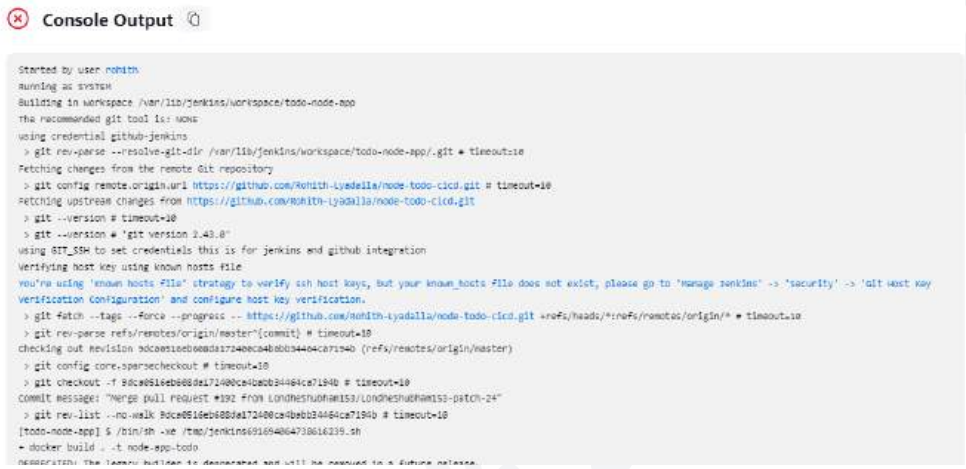
```
docker build . -t node-app-todo
docker run -d --name node-app-container -p 8000:8000 node-app-todo
```





7.3 To build the code click on Build Now



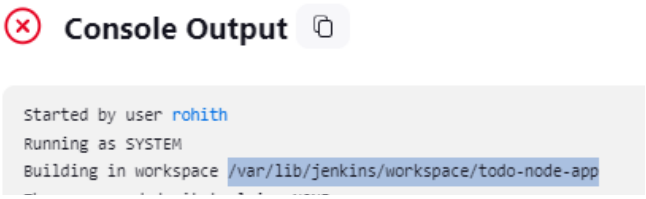


Troubleshooting:

```
permission denied while trying to connect to the Docker daemon socket at unix:///var/run/docker.sock: Post "http://%2Fvar%2Frun%2Fdocker.sock/v1.24/build?buildargs=%7B%7D&cachefrom=%5B%5D&cgroupparent=&cpuperiod=0&cpuquota=0&cpusetcpus=&cpusetmems=&cpushares=0&dockerfile=Dockerfile&labels=%7B%7D&memory=0&memse=default&rm=1&shmsize=0&t=node-app-todo&target=&ulimits=null&version=1": dial unix /var/run/docker.sock: connect: permission denied
Build step 'Execute shell' marked build as failure
Finished: FAILURE
```

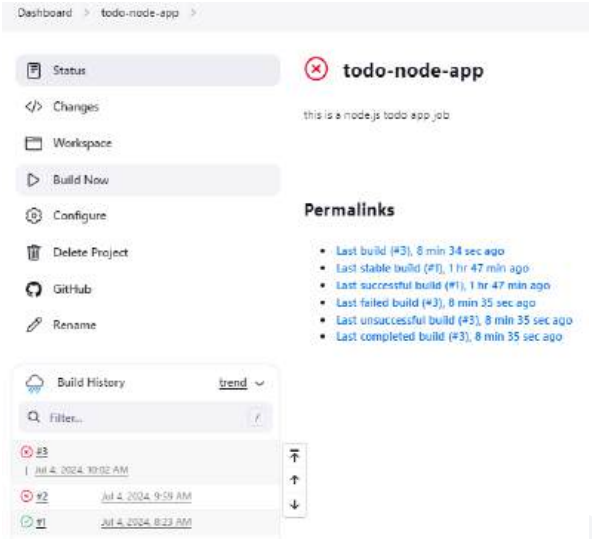
➢ Error here is, permission got dined

To resolve the problem follow the steps below

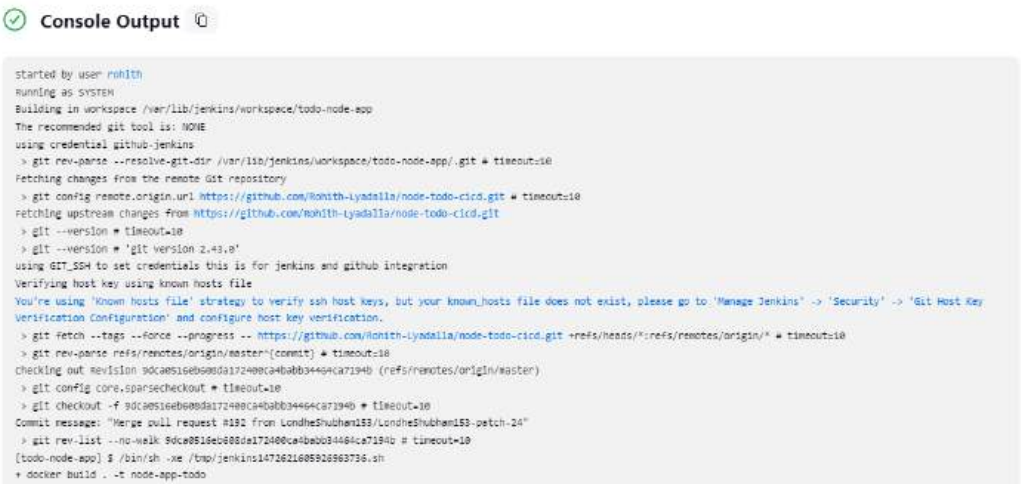



```
ubuntu@ip-172-31-80-109:/var/lib/jenkins/workspace/todo-node-app$ sudo chmod 777 /var/lib/jenkins/workspace/todo-node-app
ubuntu@ip-172-31-80-109:/var/lib/jenkins/workspace/todo-node-app$ sudo usermod -a -G docker jenkins
ubuntu@ip-172-31-80-109:/var/lib/jenkins/workspace/todo-node-app$ sudo systemctl restart jenkins
ubuntu@ip-172-31-80-109:/var/lib/jenkins/workspace/todo-node-app$
```

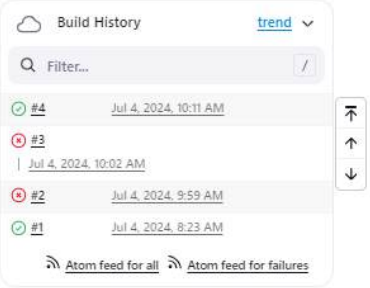
Sign in to Jenkins
Username
rohith
Password
Keep me signed in
Sign in

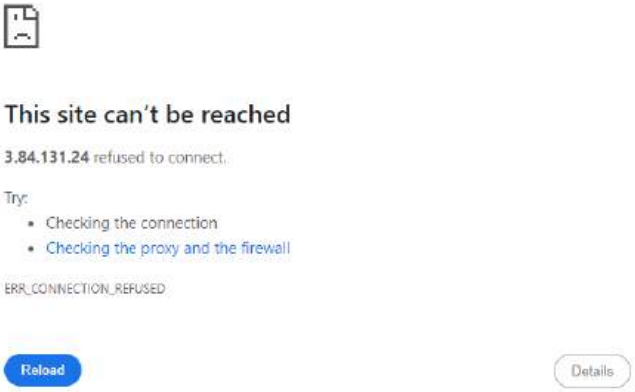

Dashboard > todo-node-app >
Status
Changes
Workspace
Build Now
Configure
Delete Project
GitHub
Rename
todo-node-app
this is a node.js todo app job
Permalinks
Last build (#3), 8 min 34 sec ago
Last stable build (#1), 1 hr 47 min ago
Last successful build (#1), 1 hr 47 min ago
Last failed build (#3), 8 min 35 sec ago
Last unsuccessful build (#3), 8 min 35 sec ago
Last completed build (#3), 8 min 35 sec ago
Build History
trend
Filter...
#3
Jul 4, 2024, 10:02 AM
#2
Jul 4, 2024, 9:59 AM
#1
Jul 4, 2024, 8:23 AM

Dashboard > todo-node-app > #4
Status
Changes
Console Output
Edit Build Information
Timings
Git Build Data
Previous Build
#4 (Jul 4, 2024, 10:11:08 AM)
Progress:
Keep this build forever
Add description
Started 0.1 sec ago
Build has been executing for 0.1 sec
No changes.
Started by user rohith
This run spent 73 ms waiting in the queue.
Revision: 9dca0516eb608da172400ca4babb34464ca7194b
Repository: https://github.com/Rohith-Lyadalla/node-todo-cicd.git
refs/remotes/origin/master


Console Output
Started by user rohith
Running as SYSTEM
Building in workspace /var/lib/jenkins/workspace/todo-node-app
The recommended git tool is: NONE
using credential github-jenkins
> git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/todo-node-app/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/Rohith-Lyadalla/node-todo-cicd.git # timeout=10
Fetching upstream changes from https://github.com/Rohith-Lyadalla/node-todo-cicd.git
> git --version # timeout=10
> git --version # 'git version 2.43.0'
using GIT_SSH to set credentials this is for jenkins and github integration
Verifying host key using known hosts file
You're using 'Known hosts file' strategy to verify ssh host keys, but your known_hosts file does not exist, please go to 'Manage Jenkins' -> 'Security' -> 'Git Host Key Verification Configuration' and configure host key verification.
> git fetch --tags --force --progress -- https://github.com/Rohith-Lyadalla/node-todo-cicd.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git rev-parse refs/remotes/origin/master^{commit} # timeout=10
Checking out Revision 9dca0516eb608da172400ca4babb34464ca7194b (refs/remotes/origin/master)
> git config core.sparsecheckout # timeout=10
> git checkout -f 9dca0516eb608da172400ca4babb34464ca7194b # timeout=10
Commit message: "Merge pull request #192 from LondheShubham153/LondheShubham153-patch-24"
> git rev-list --no-walk 9dca0516eb608da172400ca4babb34464ca7194b # timeout=10
[todo-node-app] $ /bin/sh -xe /tmp/jenkins14726216059216963736.sh
+ docker build . -t node-app-todo

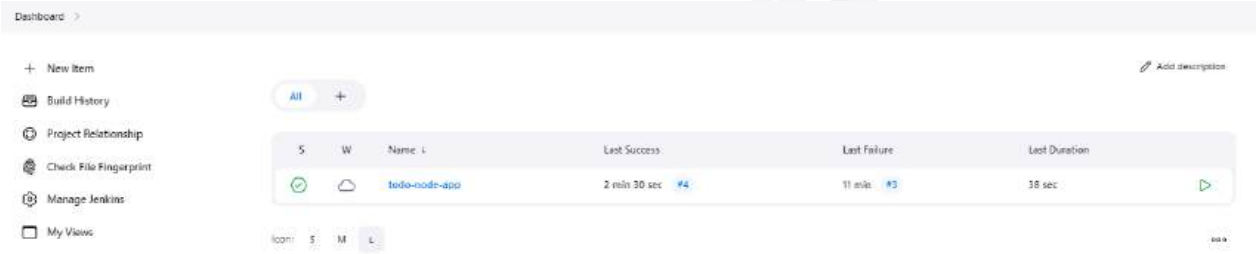
```
      ✓ Is returning 6 when multiplying 2 * 3
    Test2
executes before every test
      ✓ Is returning 4 when adding 2 + 3
executes before every test
      ✓ Is returning 8 when multiplying 2 * 4
This part executes once after all tests


  4 passing (11ms)

Removing intermediate container 1522d4085828
 ---> 66899c4b8ccd
Step 6/7 : EXPOSE 8000
 ---> Running in 63061e7fc31d
Removing intermediate container 63061e7fc31d
 ---> 4d464907edc4
Step 7/7 : CMD ["node","app.js"]
 ---> Running in 4345fc798fee
Removing intermediate container 4345fc798fee
 ---> 4bf1bde36414
Successfully built 4bf1bde36414
Successfully tagged node-app-todo:latest
+ docker run -d --name node-app-container -p 8000:8000 node-app-todo
e9eb9e39f08cf4c059a68439636d9e85d9fc1cf667c0cfae4ff1232dec7d044d
Finished: SUCCESS
```



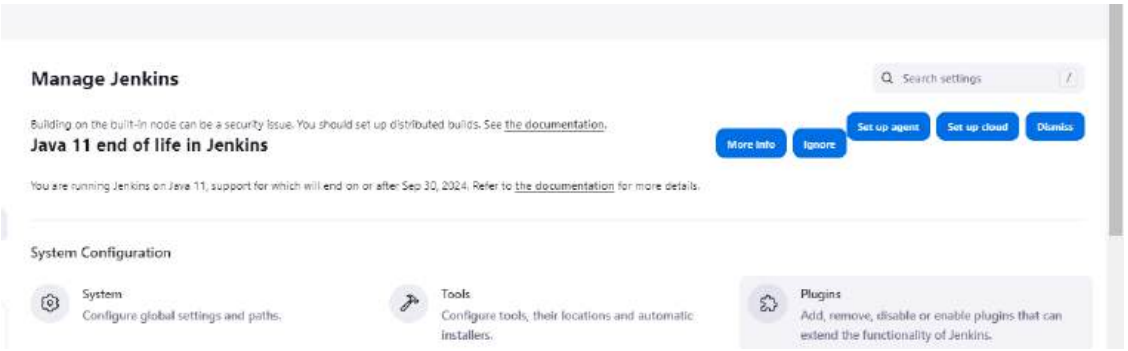


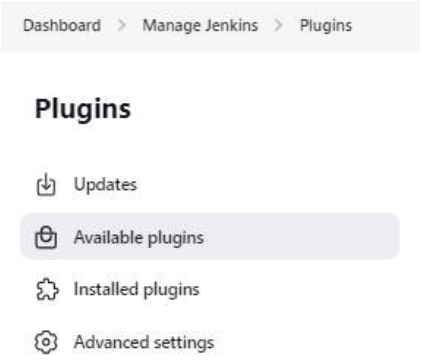
Step 8:Automating CI/CD: Integrating GitHub Webhooks with Jenkins

8.1 To automatically build the application using Jenkins. when the code is pushed into GitHub it can be done or triggered by **Webhooks,** to do that you need to download a plugin on Jenkins, for that, Go to **Jenkins Dashboard.**

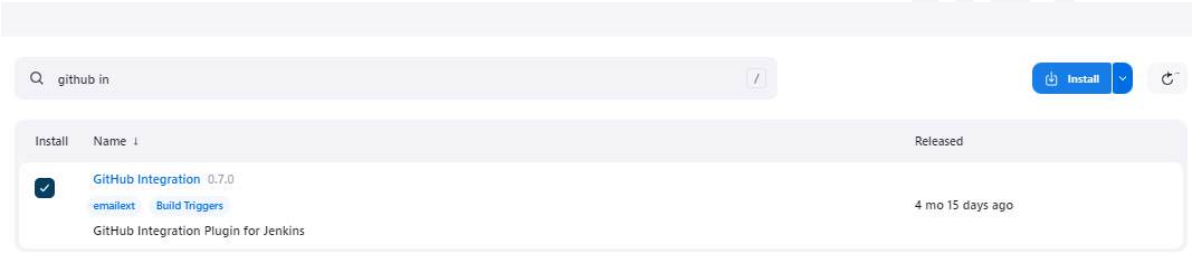


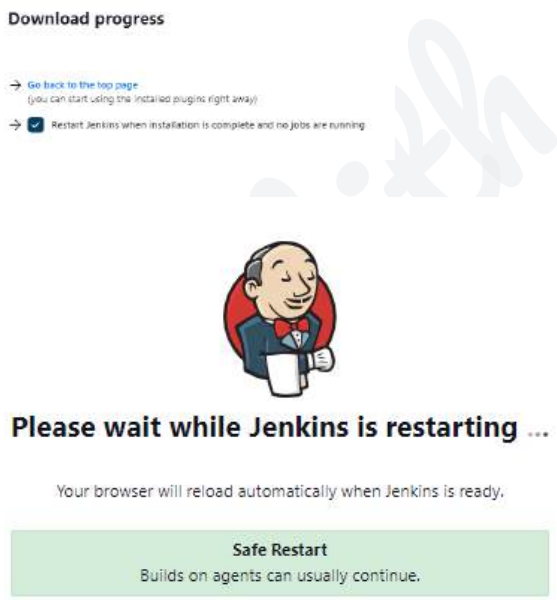
8.2 In **Jenkins Dashboard,** Click on **Manage Jenkins,** then Click on **Plugins,** Now click on **Available plugins.**





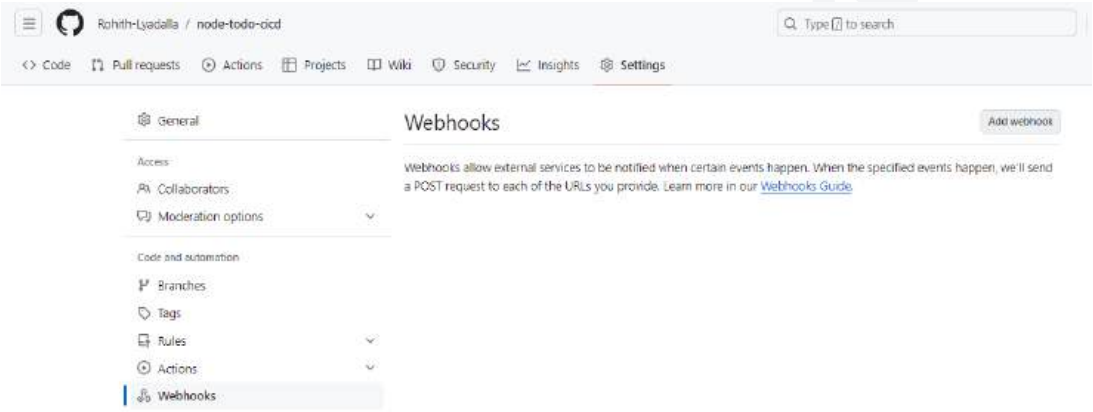
8.3 In **Available plugins,** Search for **GitHub Integration,** Click on it, then Click on **install,** now Click on **restart Jenkins when installation is complete and no jobs are running**, then Jenkins gets **restarted.**

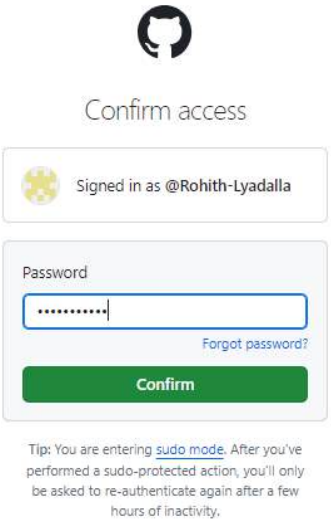




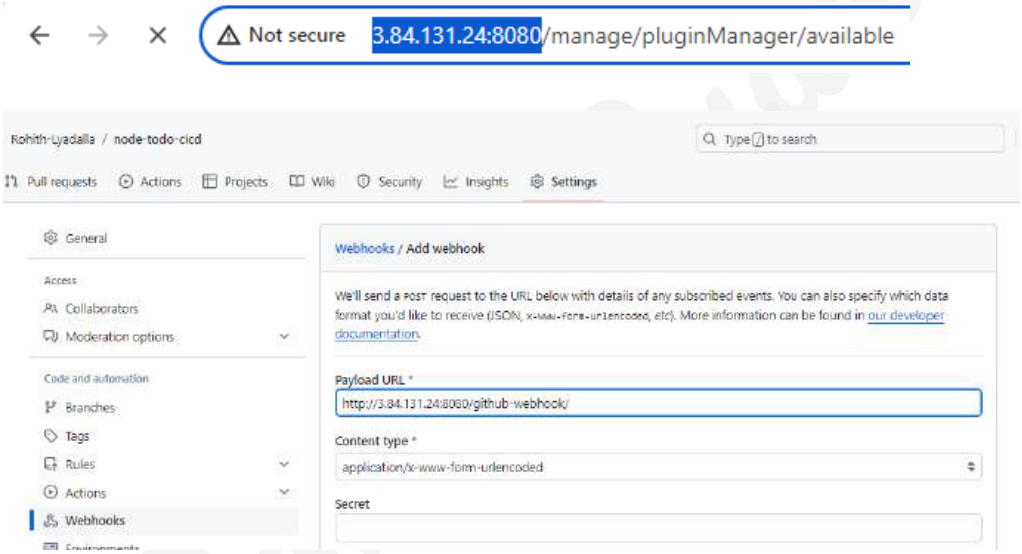
8.4 After **Jenkins restart,** enter the **Sign into Jenkins** details, such as **Username** and **Password** and then click on **Sign In.**

8.5 After Jenkins is signed in, Now go to **Github Repository,** and then go to the **Settings** section and **click on it,** Now Click on **Webhooks.** Click on Add Webhooks, then confirm access, through entering the password.

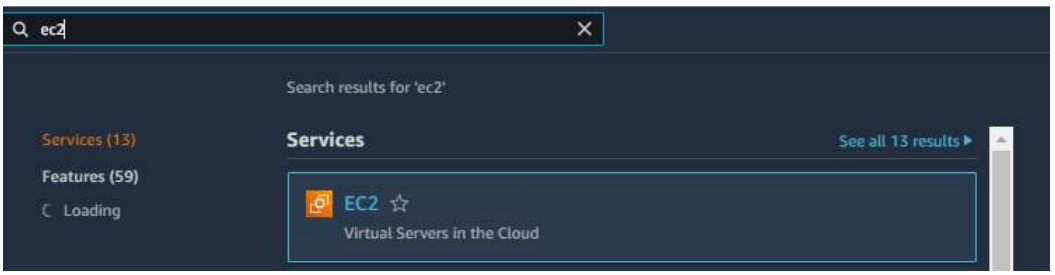


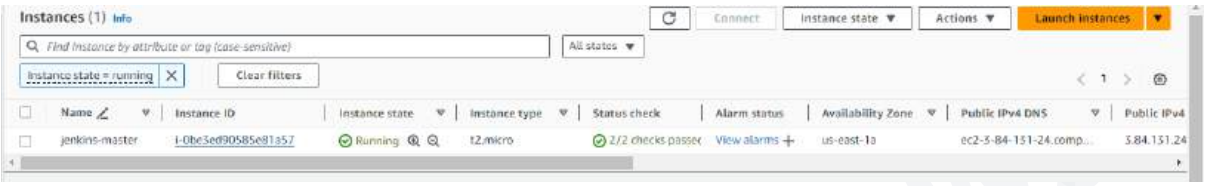


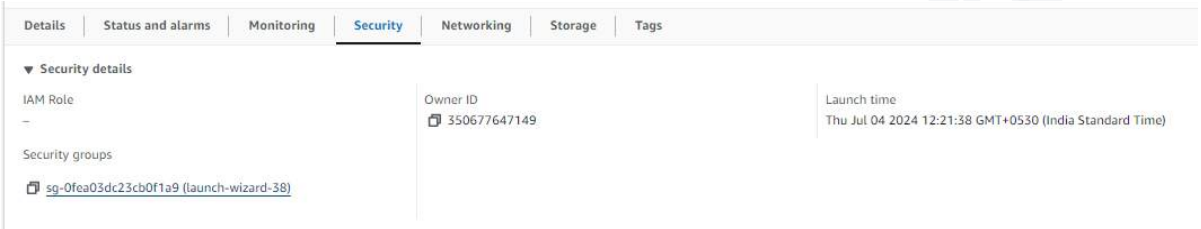
8.6 Now **Copy the payload URL of Jenkins** and **paste in on payload URL,** and **type /github-webhook/**

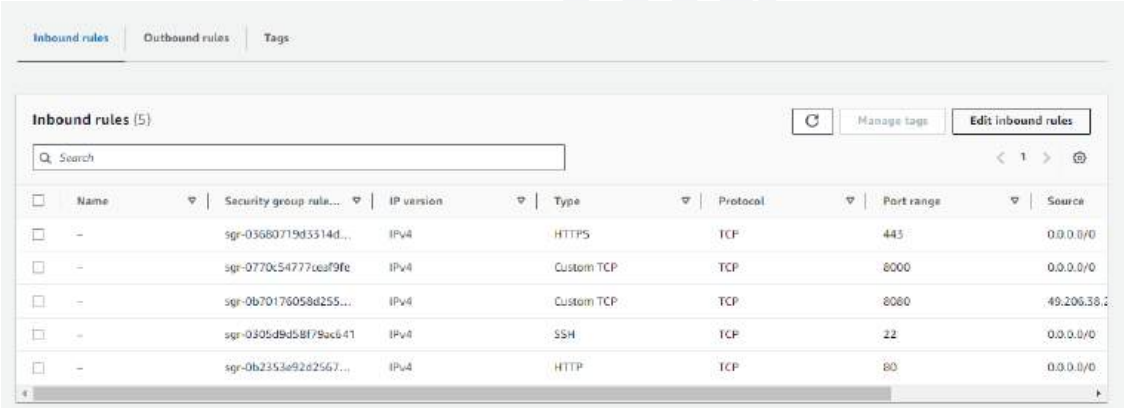


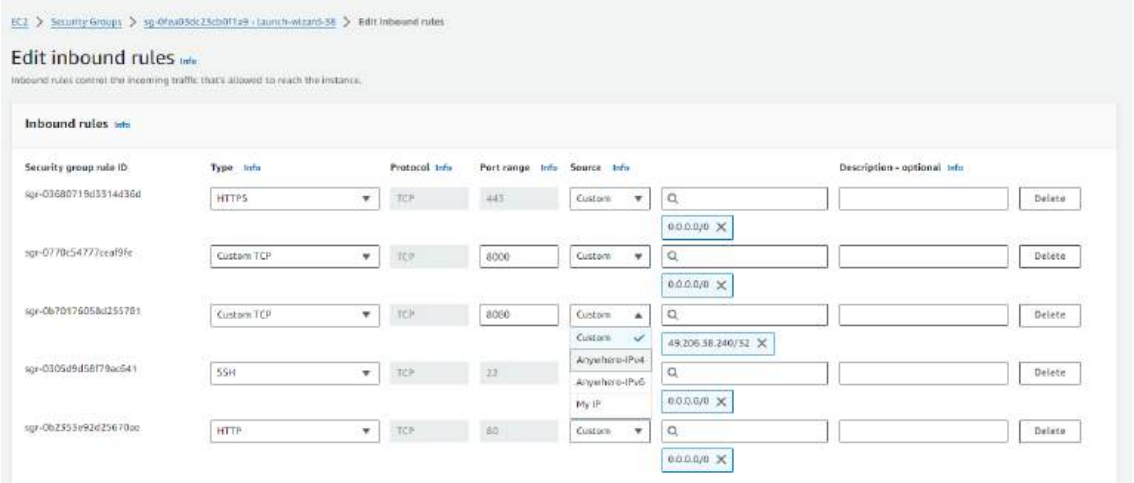
8.7 Now **PORT NO 8080** is not accessible for everyone, it is accessible by only you, to make it accessible to GitHub go to EC2 Instance, then Click on Security and Click on Security Groups, Now click on Edit Inbound Rules, now Change custom TCP rules Source from Custom to Anywhere and Save the rules.

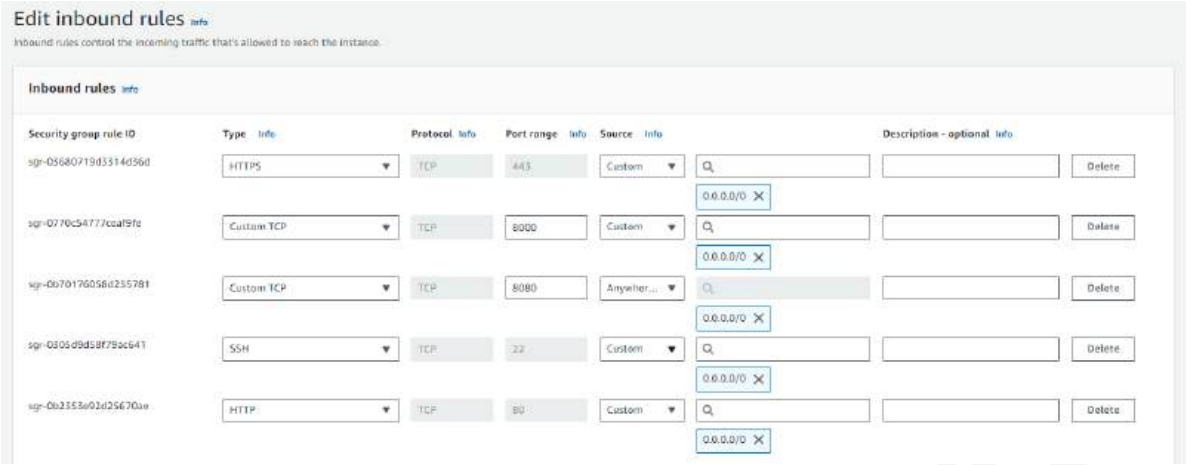

ec2
Search results for 'ec2'
Services (13)
Features (59)
Loading
Services
See all 13 results
EC2
Virtual Servers in the Cloud

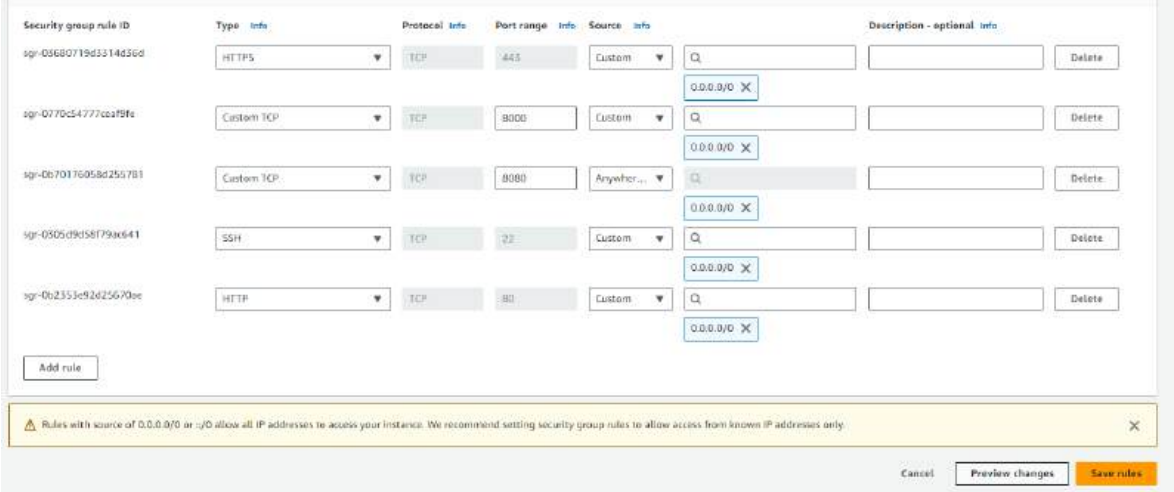

Instances (1) Info
Connect
Instance state
Actions
Launch instances
Find instance by attribute or tag (case-sensitive)
All states
Instance state = running
Clear filters
Name
Instance ID
Instance state
Instance type
Status check
Alarm status
Availability Zone
Public IPv4 DNS
Public IPv4
jenkins-master
i-0be3ed90585e81357
Running
t2.micro
2/2 checks passed
View alarms
us-east-1a
ec2-3-84-151-24.comp...
3.84.151.24

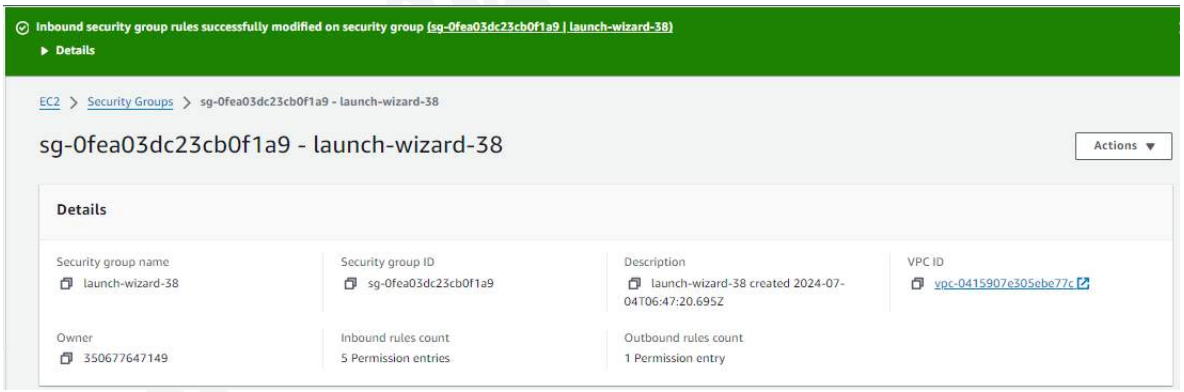

Details
Status and alarms
Monitoring
Security
Networking
Storage
Tags
Security details
IAM Role
–
Owner ID
350677647149
Launch time
Thu Jul 04 2024 12:21:38 GMT+0530 (India Standard Time)
Security groups
sg-0fea03dc23cb0f1a9 (launch-wizard-38)


Inbound rules
Outbound rules
Tags
Inbound rules (5)
Manage tags
Edit inbound rules
Search
Name
Security group rule...
IP version
Type
Protocol
Port range
Source
sgr-03680719d3314d...
IPv4
HTTPS
TCP
443
0.0.0.0/0
sgr-0770c54777ceaf9fe
IPv4
Custom TCP
TCP
8000
0.0.0.0/0
sgr-0b70176058d255...
IPv4
Custom TCP
TCP
8080
49.206.38.
sgr-0305d9d58f79ac641
IPv4
SSH
TCP
22
0.0.0.0/0
sgr-0b2353e92d2567...
IPv4
HTTP
TCP
80
0.0.0.0/0


EC2 > Security Groups > sg-0fea03dc23cb0f1a9 - launch-wizard-38 > Edit inbound rules
Edit inbound rules Info
Inbound rules control the incoming traffic that's allowed to reach the instance.
Inbound rules Info
Security group rule ID
Type Info
Protocol Info
Port range Info
Source Info
Description - optional Info
sgr-03680719d3314d36d
HTTPS
TCP
443
Custom
0.0.0.0/0
Delete
sgr-0770c54777ceaf9fe
Custom TCP
TCP
8000
Custom
0.0.0.0/0
Delete
sgr-0b70176058d255781
Custom TCP
TCP
8080
Custom
Custom
49.206.38.240/32
Delete
sgr-0305d9d58f79ac641
SSH
TCP
22
Anywhere-IPv4
Anywhere-IPv6
My IP
0.0.0.0/0
Delete
sgr-0b2353e92d25670ac
HTTP
TCP
80
Custom
0.0.0.0/0
Delete







8.8 Now in Content type, change it to, application json, because our application is in JSON format, Now Click on Add Webhook, then reload the page and then Webhook will be integrated with GitHUB

Webhooks / Add webhook

We'll send a POST request to the URL below with details of any subscribed events. You can also specify which data format you'd like to receive (JSON, x-www-form-urlencoded, etc). More information can be found in our developer documentation.

Payload URL *

http://3.84.131.24:8080/github-webhook/

Content type *

application/json

Rules
Actions
Webhooks
Environments
Codespaces
Pages
Security
Code security and analysis
Deploy keys
Secrets and variables
Integrations
GitHub Apps
Email notifications

application/json

Secret

SSL verification

By default, we verify SSL certificates when delivering payloads.

Enable SSL verification Disable (not recommended)

Which events would you like to trigger this webhook?

Just the push event.

Send me everything.

Let me select individual events.

Active

We will deliver event details when this hook is triggered.

Add webhook

Type / to search

Wiki Security Insights Settings

est it out! Read more about it at https://docs.github.com/webhooks/#ping-event.

Webhooks

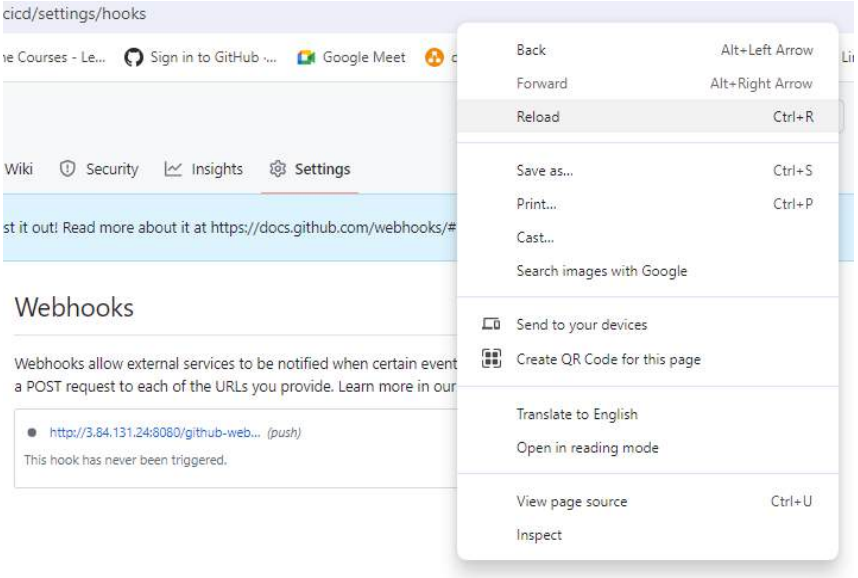
Add webhook

Webhooks allow external services to be notified when certain events happen. When the specified events happen, we'll send a POST request to each of the URLs you provide. Learn more in our Webhooks Guide.

http://3.84.131.24:8080/github-web... (push)

This hook has never been triggered.

Edit Delete



8.9 After integrating Webhook with GitHub, whenever we make any changes in the Code of the application, Webhook will Automatically trigger from GitHub and then the Code is built on Jenkins Automatically, without manual intervention.

Conclusion:

The integration of Jenkins with GitHub provides a robust solution for automating the Continuous Integration and Continuous Delivery (CI/CD) pipeline, significantly enhancing the efficiency of software development workflows. By automating build, test, and deployment processes, this integration enables teams to detect and resolve issues early, maintain code quality, and deliver features rapidly and reliably.

Rohith Gyadella